COMPANION TECHNICAL WHITEPAPER

Implementing Exclusive Domain-to-CA Binding in Chromium

Custom verifier architecture for TCP TLS, QUIC/HTTP/3 and locally added private trust

Private ungoogled-chromium fork • Linux and macOS build runbooks • signed central policy • CA rotation • domain onboarding

Core design property A domain rule changes certificate acceptance only when the canonical reference hostname matches that rule. For every unmatched hostname, ordinary Chromium trust—including public roots, platform roots and locally added private roots—remains unchanged. QUIC uses the same policy result rather than treating every private root as unknown.

Version 1.0 | 26 July 2026

Companion to “Constrained Certificate Trust in Chrome”, version 2.1.

Document control

Table 1 — Document status and scope

Field	Value
Title	Implementing Exclusive Domain-to-CA Binding in Chromium
Document type	Technical architecture whitepaper and implementation runbook
Version	1.0
Date	26 July 2026
Target platforms	Linux x86-64/arm64 and macOS arm64/x86-64
Reference protected namespace	scone.cloud, including api.scone.cloud and deeper subdomains
Reference CA identity	Approved SHA-256 SPKI hashes and optional DER certificates; ca.scone.cloud is a descriptive label
Recommended browser base	Private patch-stack fork of ungoogled-chromium, with portable-Linux and macOS packaging forks
Primary implementation	Request-scoped trust-anchor filtering inside Chromium's shared verifier, a final-chain postcondition, and an explicit QUIC private-root disposition
Primary conclusion	Selective exclusive CA binding is implementable without disabling QUIC or private trust globally. Unmatched domains retain baseline Chromium behavior.

Accuracy boundary This report is an implementation design grounded in current Chromium and ungoogled-chromium source structure. Chromium changes continuously; exact file names and call sites must be revalidated for the pinned Chromium milestone before each implementation or rebase.

Abstract

The companion whitepaper “Constrained Certificate Trust in Chrome” established that stock Chrome can add a CA constrained to a DNS subtree but cannot require that the validated path for that subtree contain only an explicitly approved CA. This report specifies how to add that missing property to a private Chromium-derived browser. It covers base-browser selection, fork organization, a concrete enterprise policy schema, the verifier algorithm, QUIC and HTTP/3 behavior, locally added private trust, session and connection reuse, dynamic updates, secure centralized policy distribution, CA key rotation, delegated domain onboarding, Linux and macOS builds, automated testing, release engineering and an issue-ready implementation backlog.

The recommended design filters eligible trust anchors before certification-path construction. A request-scoped trust-store wrapper examines the canonical reference hostname. When no domain rule matches, it delegates every trust decision unchanged to Chromium’s aggregate public, platform and enterprise trust store. When a rule matches, it makes only approved CA SPKIs eligible as trust anchors while preserving explicit distrust. Chromium then performs its normal RFC 5280 path building, hostname, algorithm, revocation and Certificate Transparency checks. A final postcondition verifies that the selected chain contains the approved anchor. TCP TLS and QUIC both consume the same result. The QUIC proof verifier is additionally taught to distinguish a locally trusted private root from an arbitrary unknown root, so private trust can remain available for unconstrained domains without weakening constrained domains.

Executive summary

A custom verifier can provide the security property that stock Chrome lacks while preserving normal browsing elsewhere. The key is to avoid modifying the global trust store and avoid applying a universal pin. Before path construction, the browser selects a request-scoped view of the trust store. For scone.cloud, only organization-approved CA SPKIs are eligible as anchors. For unrelated.example, no rule matches, so Chrome Root Store trust, enterprise roots and locally installed private roots continue to work exactly as before. A defense-in-depth postcondition checks the final chain as well.

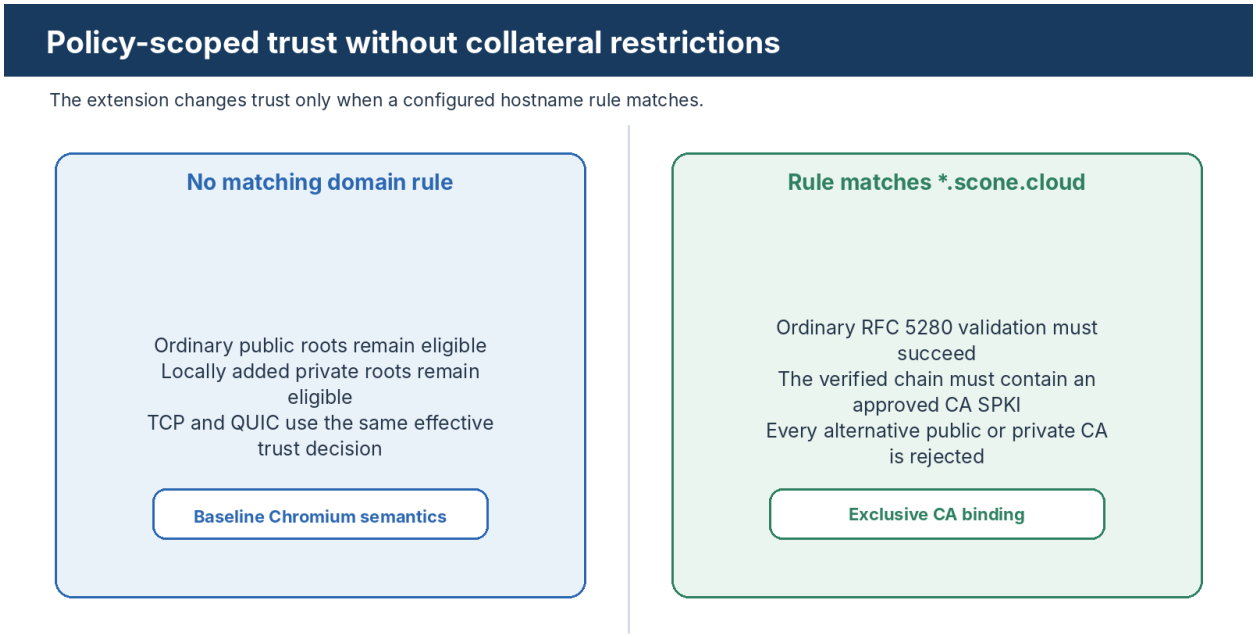


Figure 1 — Policy-scoped trust without collateral restrictions.

Table 2 — Principal design decisions

Decision	Recommendation	Reason
Chromium base	Use ungoogled-chromium as a patch-stack and packaging foundation; keep the security feature as a small, independently reviewable patch series.	It remains close to upstream Chromium and already separates common patches from platform packaging.
Policy shape	Introduce one atomic DomainTrustPolicy dictionary containing authorities, rules, generation and defaults.	An anchor and the rule that restricts it must not arrive from different sources or at different times.
CA identity	Use SHA-256 SPKI hashes; optionally carry DER CA certificates for private trust bootstrap.	SPKI survives controlled certificate reissuance and is directly available from CertVerifyResult.
Rule matching	Canonical IDNA A-label hostname, complete-label suffix matching, longest suffix wins.	Prevents suffix confusion and permits safe delegation of narrower namespaces.
Verifier location	Add request-scoped trust-anchor filtering before path construction, plus a final-chain postcondition.	Filtering avoids false rejection when an approved alternate path exists; the postcondition detects integration mistakes.
QUIC	Keep QUIC enabled. Replace the private-root rejection heuristic with an explicit verifier disposition.	Matched approved private roots and unmatched locally trusted private roots can be accepted without weakening constrained domains.
Unmatched domains	Return the underlying Chromium result and preserve all existing trust sources.	The extension must not become a new global trust policy.
Updates	Distribute signed, versioned, expiring policy bundles with rollback protection and last-known-good state.	HTTPS transport alone must not authorize trust-policy changes.

Decision	Recommendation	Reason
CA rotation	Publish old and new SPKIs together, wait for client adoption, change servers, then remove old.	Avoids fail-open shortcuts and permits bounded overlap.
Domain onboarding	Use a central policy authority with constrained delegated signing keys per DNS subtree.	A tenant can manage its own CA without gaining authority over another tenant's namespace.

Recommended implementation boundary Do not remove public roots globally and do not rely on a post-verification pin alone. Filter trust anchors for matching hostnames before path construction, keep Chromium's normal validation inside that filtered view, verify the final anchor as a postcondition, and propagate the policy generation and trust disposition to connection reuse and QUIC.

Contents

Table 3 — Contents

Section	Title	Page
1	Requirements, threat model and invariants	7
2	Selecting an open-source Chromium base	9
3	Private ungoogled-chromium fork and repository topology	11
4	Chromium verifier and transport architecture	13
5	DomainTrustPolicy configuration extension	15
6	Verifier algorithm and source integration	19
7	QUIC, HTTP/3 and locally added private trust	23
8	Connection reuse, resumption and dynamic updates	25
9	Secure centralized configuration	27
10	Automated CA rotation and domain onboarding	29
11	Build and development runbooks	32
12	Test strategy and conformance laboratory	35
13	CI, release engineering and operations	38
14	Security analysis and residual risk	40
15	Implementation roadmap and conclusions	42
A	Complete policy examples	43
B	Primary source-code integration map	45
C	Issue-ready implementation backlog	46
D	Conformance test matrix	62
E	Command and release runbook	64
F	References	66

1. Requirements, threat model and invariants

1.1 Security objective

For a protected DNS subtree such as scone.cloud, the extended browser shall accept a TLS server certificate only when ordinary Chromium validation succeeds and the certification path terminates at an explicitly approved trust anchor. A chain to a different Chrome Root Store CA, a different platform root or a different enterprise-added root must be rejected for that subtree even when that chain would otherwise be valid. For every hostname outside all configured subtrees, certificate verification must remain behaviorally equivalent to the selected Chromium baseline.

NORMATIVE ACCEPTANCE RULE

```
For reference hostname h and policy snapshot P:

if MatchRule(P, Canonicalize(h)) = none:
   return ChromiumVerify(h, baseline_trust_store)

allowed = Anchors(MatchRule(P, h))
result = ChromiumVerify(h, Filter(baseline_trust_store, allowed))
require result == OK
require FinalAnchorSPKI(result) in allowed
return OK
```

The policy is a relying-party restriction. It does not replace normal hostname validation, certificate validity, algorithm constraints, revocation policy, Certificate Transparency requirements or explicit distrust. It only narrows which trust anchors may complete an otherwise valid path for a matching hostname.

1.2 Threat model

Table 4 — In-scope adversaries and expected result

Threat	Capability	Required browser result
Unexpected public-CA issuance	The adversary obtains a browser-valid certificate for api.scone.cloud from a public CA that is not approved by the domain rule.	Reject because the path cannot terminate at an eligible anchor.
Enterprise TLS interception	A managed endpoint trusts an enterprise interception CA that signs a certificate for the protected service.	Reject for the protected subtree unless that CA SPKI is explicitly approved; remain usable for unrelated domains.
Locally installed private CA	A user or device-management system adds a private trust anchor.	Preserve baseline trust for unmatched domains; reject it for matched domains unless explicitly approved.
Alternative/cross-signed path	The server, AIA cache or platform offers both approved and unapproved paths.	Build an approved path when one exists; do not reject merely because an unapproved path was considered first.
QUIC-specific unknown-root behavior	HTTP/3 reaches Chromium's additional known-root check after ordinary verification.	Accept approved or legitimately locally trusted private roots according to the same domain rule; reject arbitrary unknown roots.
Policy rollback	An attacker replays an older signed bundle that re-authorizes an old CA.	Reject by generation/epoch monotonicity and persisted rollback state.
Policy-update outage	The central service is unreachable.	Use the last-known-good unexpired snapshot. Fail closed only for matching protected subtrees when the snapshot expires.
Malicious web content	A page, extension or server response tries to weaken the restriction.	No effect. Only mandatory machine policy and the independently signed bundle can change the rule.

1.3 Non-goals and assumptions

- The browser does not prove that an approved CA issued a certificate only after remote attestation. Such issuance policy belongs to the CA or attestation-aware ACME service.
- The design does not protect an endpoint on which the attacker can replace the signed browser binary, replace the configured policy root key and suppress integrity monitoring.
- The design does not make Certificate Transparency or CAA unnecessary. They remain valuable issuance controls and detection mechanisms.

- The design does not permit a website to self-pin through a response header. Domain rules are centrally governed and revocable.
- The first implementation targets desktop Linux and macOS. The architecture is intentionally compatible with later Windows and Android work, but their packaging and platform trust integrations require separate qualification.

1.4 Normative invariants

Table 5 — Invariants that implementation and tests must preserve

ID	Invariant
I-1	No matching rule means no change to trust-anchor eligibility or the underlying verification result.
I-2	The most-specific complete-label DNS subtree match wins; raw string suffix matching is prohibited.
I-3	A matching rule narrows eligible anchors before path construction, not only after a path has been selected.
I-4	Explicit distrust from Chromium, platform or enterprise policy always overrides authorization by a domain rule.
I-5	A successful matching-domain result records the matched rule ID, policy generation, final trust-anchor SPKI and trust-anchor source.
I-6	TCP TLS and QUIC use the same domain policy snapshot and the same trust decision.
I-7	A locally trusted private anchor remains usable over QUIC for an unmatched domain, but an arbitrary unknown root does not.
I-8	Tightening a rule cannot be bypassed through pooled connections, TLS resumption, QUIC connection reuse or stale verifier cache entries.
I-9	A malformed or partially verified new policy snapshot never replaces the last-known-good snapshot.
I-10	Policy and CA rollovers use explicit overlap; no implementation path temporarily disables enforcement.

Why filtering must precede path construction A post-verification SPKI check alone can falsely reject a server when Chromium selected an unapproved path even though an approved alternative path exists. Request-scoped trust-store filtering removes unapproved anchors from eligibility, allowing the ordinary path builder to discover an approved path. The final SPKI check remains as defense in depth.

2. Selecting an open-source Chromium base

Google Chrome is proprietary in important packaging, update and service-integration layers, but its browser engine and network stack are developed in the open-source Chromium project. Several browsers and distributions build on Chromium. The relevant selection criterion is not merely openness: the project must be maintainable as a security-sensitive private fork, support Linux and macOS, retain access to Chromium's net stack and tests, and provide a disciplined method for carrying a small patch series.

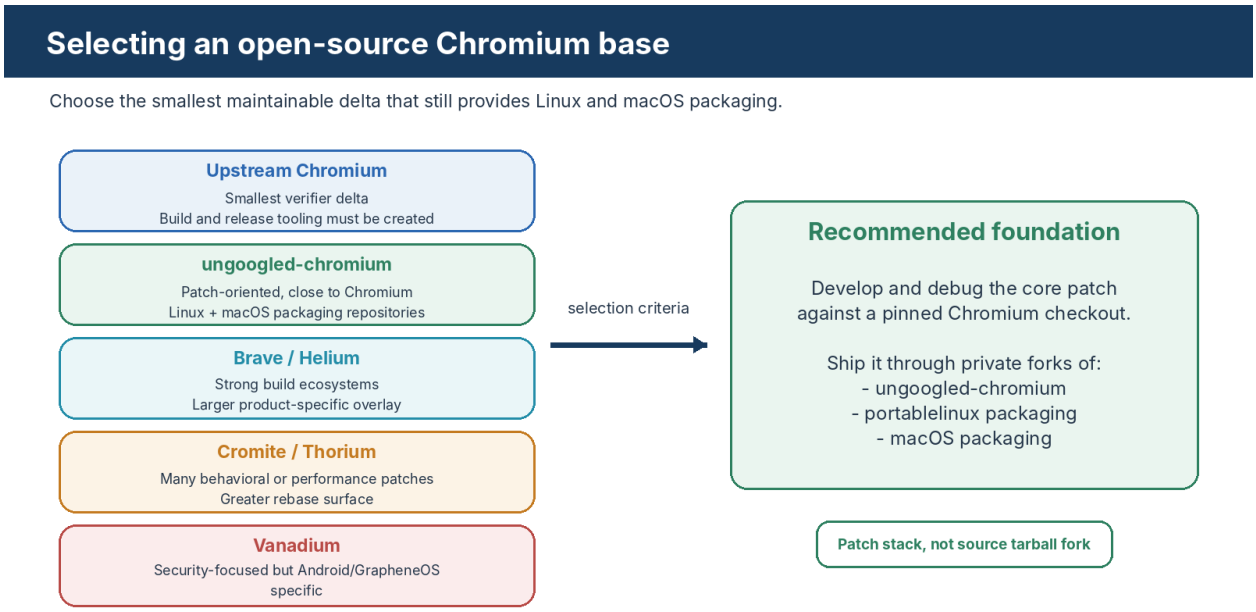


Figure 2 — Recommended base selection and productization path.

Table 6 — Representative open-source Chromium variants

Project	Platforms relevant here	Extension model	Assessment for verifier fork	Recommendation
Chromium	Linux, macOS and others	Direct source modification in the monorepo.	Best source of truth and fastest place to prototype; packaging, branding and update infrastructure remain organization responsibilities.	Use for architecture spikes and upstream tracking.
ungoogled-chromium	Common patch set plus platform packaging repos, including portable Linux and macOS	GNU Quilt patch series, build flags and source processors layered over Chromium.	Good fit for a private downstream because verifier changes can be isolated in an auditable patch group and rebased with the existing patch workflow. [1][2]	Recommended product foundation.
Brave	Linux, macOS, Windows, mobile	Large product layer and generated/overlay changes around Chromium.	Open source and well engineered, but carries a larger product-specific delta and service architecture than required for a dedicated trust-verifier browser.	Consider only if Brave product features are required.
Cromite	Primarily Android; current project also publishes selected desktop builds	Security/privacy patch set over Chromium.	Valuable source of patches, but desktop/macOS production scope and packaging are less aligned with this project.	Not the primary base.
Thorium / Helium / Iridium	Varies by project	Downstream patch sets and packaging.	Potentially useful references, but smaller project-specific release and maintenance models add supply-chain and longevity risk.	Evaluate as references, not default base.
Vanadium	GrapheneOS/Android	Hardened Chromium for GrapheneOS.	Strong security orientation, but platform-specific and not a Linux/macOS desktop base.	Out of scope for first release.

2.1 Why ungoogled-chromium is the practical downstream

ungoogled-chromium separates common configuration and patches from platform-specific packaging. Its design documentation identifies configuration files, source processors, patches and packaging as distinct components.

Patches are maintained in GNU Quilt format under patches/series and are required to apply without fuzz. Platform repositories consume the common repository and add the build and packaging logic required for a specific target. This structure is suitable for a private security fork because the custom verifier can live in a clearly named patch group rather than becoming an unreviewable long-lived monorepo divergence. [1][2]

- Keep the verifier feature independent of ungoogled privacy changes: its patch group should apply to a clean, pinned Chromium source tree in CI.
- Retain a direct upstream-Chromium prototype branch so architectural failures can be diagnosed without the complete downstream patch stack.
- Treat ungoogled-chromium as packaging and downstream-maintenance machinery, not as a security boundary. Review every inherited patch and build flag.
- Do not assume downstream README toolchain versions remain authoritative for a future Chromium milestone. The pinned upstream milestone defines supported Clang, Rust, Xcode and SDK requirements.

2.2 Why a browser extension is insufficient

An extension executes above the certificate verifier. It may observe or block many requests, but it cannot make trust-anchor selection part of every TCP TLS and QUIC path build, cannot reliably control browser-internal requests, and cannot safely invalidate all pooled or resumed connections. The enforcement point must remain in the native network stack. An extension may display diagnostics or export telemetry, but it must not be the sole security boundary.

2.3 Licensing and distribution considerations

Chromium and the cited downstream projects contain code under multiple permissive and third-party licenses. A product fork must preserve notices, generate a reproducible third-party license bundle, publish source where required by an incorporated component, and separate organization-owned policy-controller code from browser code when different licensing or disclosure obligations apply. The legal review should also cover branding: a private browser must not imply that it is Google Chrome.

3. Private ungoogled-chromium fork and repository topology

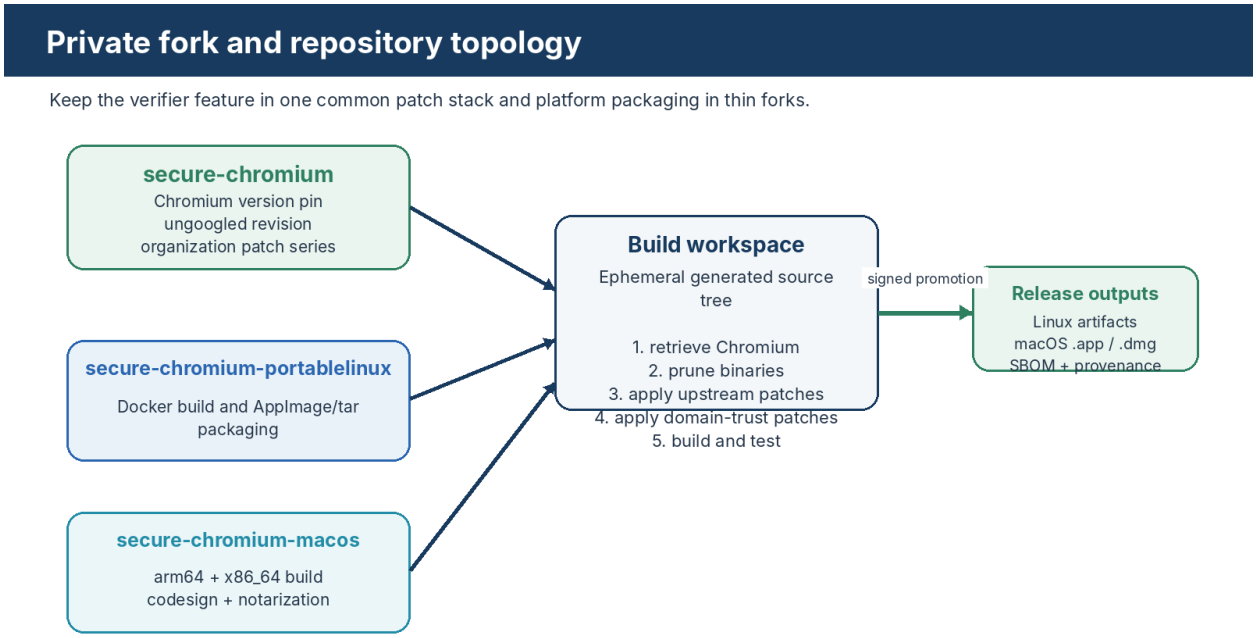


Figure 3 — Recommended private repository topology.

3.1 Repository layout

Table 7 — Repositories and responsibilities

Repository	Purpose	Protected branches / artifacts
browser-common	Private fork of ungoogled-chromium common configuration. Contains patches/secure-domain-trust, policy schema, feature flags and shared test data.	vendor/<milestone>, feature/*, release/<milestone>; signed tags.
browser-portablelinux	Private fork of ungoogled-chromium-portablelinux. Pins browser-common as a submodule and produces AppImage/tar packages plus symbols.	release/*; signed AppImage/tar, SBOM, provenance and test report.
browser-macos	Private fork of ungoogled-chromium-macos. Pins browser-common and produces signed/notarized DMG or PKG artifacts.	release/*; notarized app/DMG, SBOM, provenance and test report.
domain-trust-controller	Validates onboarding requests, signs policy bundles, publishes update components, manages delegations and audit history.	main plus environment branches; signed container images and policy bundles.
domain-trust-test-pki	Reproducible roots, intermediates, cross-signs, revocation responders and test servers for positive/negative cases.	Immutable scenario tags; test-only keys clearly separated from production.
chromium-upstream-mirror	Optional read-only mirror or cache of the pinned Chromium source and component artifacts.	No private changes; integrity-checked mirror.

3.2 Remote and branch policy

REPOSITORY BOOTSTRAP

```
# browser-common
git remote add upstream https://github.com/ungoogled-software/ungoogled-chromium.git
git remote add chromium https://chromium.googlesource.com/chromium/src.git
git remote set-url --push upstream DISABLED
git remote set-url --push chromium DISABLED

# Example branch naming
vendor/136.0.7103.XX-1
feature/domain-trust-policy
feature/quic-local-anchor-disposition
release/136.0.7103.XX-scone.1
```

The vendor branch is an unmodified import of the selected downstream release. Feature branches contain reviewable changes. A release branch is produced only by the automated rebase pipeline after patch validation and conformance tests. No developer should commit directly to vendor or release branches.

3.3 Patch organization

PRIVATE PATCH SERIES

```
patches/secure-domain-trust/
  0001-policy-schema-and-prefs.patch
  0002-policy-bundle-model-and-parser.patch
  0003-request-scoped-trust-store-filter.patch
  0004-cert-verify-result-trust-disposition.patch
  0005-quic-local-private-root-support.patch
  0006-session-and-cache-invalidation.patch
  0007-diagnostics-and-netlog.patch
  0008-component-updater-policy-source.patch
  0009-unit-and-browser-tests.patch

# Append after inherited patches in patches/series:
secure-domain-trust/0001-policy-schema-and-prefs.patch
...
secure-domain-trust/0009-unit-and-browser-tests.patch
```

Each patch should compile and, where practical, carry its own tests. The series order is part of the reviewed interface. Patches must apply with no fuzz, matching the ungoogled-chromium maintenance rule. [1] A generated patch may be acceptable for schema outputs, but its generator input must be in an earlier patch and CI must prove regeneration is clean.

3.4 Rebase workflow

1. Import and sign the upstream ungoogled-chromium release tag and its pinned Chromium version.
2. Update the browser-common submodule revision in each packaging repository on a temporary rebase branch.
3. Merge the platform patch series when required by the packaging repository, then apply the private patch group with Quilt.
4. Resolve failures one patch at a time. Do not use patch fuzz or broad context reduction to make a security-sensitive patch appear to apply.
5. Run source-level unit tests and the complete test-PKI conformance suite before package generation.
6. Build Linux and macOS artifacts from clean workers, sign them, generate SBOM/provenance, and promote first to a pilot channel.
7. Record Chromium commit, ungoogled revisions, patch hashes, compiler/toolchain identifiers and policy-schema version in the release manifest.

3.5 Supply-chain controls

- Pin every source archive, submodule and prebuilt tool by cryptographic digest. Mirror artifacts only after verification against the authoritative source.
- Require two-person review for changes to trust semantics, updater keys, code-signing identities and release-channel configuration.
- Build on ephemeral workers with network egress restricted to a pre-populated dependency mirror.
- Generate SLSA-style provenance or an equivalent signed build attestation that binds the output to source revisions and build parameters.
- Store macOS Developer ID credentials, package-signing keys and policy-signing keys in separate hardware-backed identities.

4. Chromium verifier and transport architecture

Chromium's network stack deliberately centralizes certificate verification behind net::CertVerifier. TCP-based TLS in SSLClientSocketImpl constructs CertVerifier::RequestParams from the server certificate, reference hostname, OCSP/SCT data and flags. QUIC's ProofVerifierChromium also invokes the certificate verifier and then applies additional transport-specific checks. Enterprise certificate settings are assembled by ProfileNetworkContextService and sent to the verifier service. Exact class and Mojo boundaries can move between milestones, but these responsibilities are stable integration landmarks. [6][7][8]

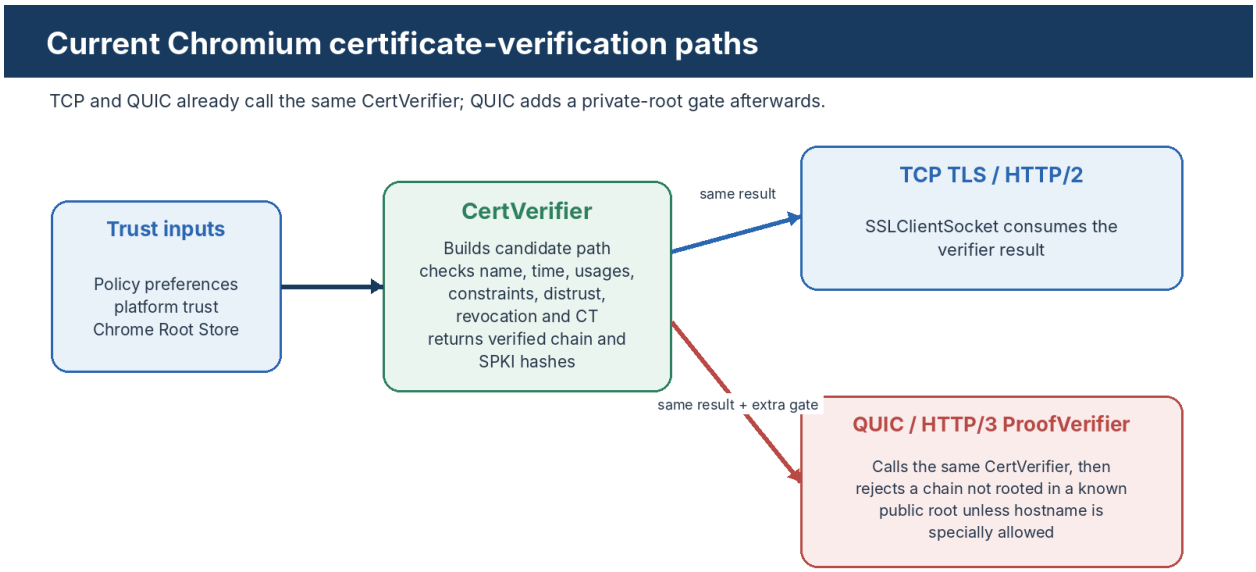


Figure 4 — Current TCP TLS and QUIC certificate-verification paths.

4.1 Trust sources and path construction

The built-in verifier constructs paths using an aggregate trust view that can include the Chrome Root Store, platform or user trust, enterprise-added anchors, additional intermediates and explicit distrust. Existing enterprise constrained trust is represented as additional anchors with permitted DNS/CIDR constraints. ProfileNetworkContextService serializes these inputs to the verifier service; it does not select an exclusive anchor set based on the request hostname. [8]

Table 8 — Trust inputs that the filter must preserve

Input	Unmatched hostname	Matched hostname
Chrome Root Store anchor	Eligible exactly as in baseline Chromium.	Eligible only when its SPKI is approved by the matched rule.
Platform/user private anchor	Eligible exactly as in baseline Chromium.	Eligible only when approved; still subject to platform distrust and constraints.
Enterprise-added anchor	Eligible exactly as in baseline Chromium.	Eligible only when approved; existing Name Constraints remain enforced.
Intermediate certificate	Available as an issuer hint; not automatically trusted.	Still available for building an approved path.
Explicitly distrusted SPKI	Distrusted.	Distrusted; the domain rule cannot override it.
Existing constrained anchor	Eligible when its existing constraints permit the name.	Must satisfy both the existing constraints and the new domain rule.

4.2 Why a final-chain-only check is insufficient

Consider a leaf and cross-signed intermediate that admit two valid paths: one to approved private root A and one to unapproved public root B. If ordinary path construction chooses B, a final-chain-only pin rejects the connection even though A could have produced a compliant path. The desired semantics are not “verify normally, then complain”; they are “verify normally within the set of anchors authorized for this hostname.” Therefore the primary control must sit at the trust-store interface consulted by path construction. The final-chain SPKI comparison is retained only as a postcondition.

4.3 Integration choices

Table 9 — Candidate integration points

Option	Advantages	Disadvantages	Decision
Modify every TCP and QUIC caller	Direct visibility of hostname and transport.	Duplicates security logic; misses future callers; difficult to keep consistent.	Reject.
Post-process CertVerifier result only	Small patch; easy to prototype.	Can falsely reject when an approved alternate path exists; does not constrain path discovery.	Use only as an early prototype and defense-in-depth check.
Request-scoped trust-store filter inside built-in verification	Correct anchor eligibility before path building; all callers benefit.	Touches verifier internals and must follow BoringSSL/Chromium trust-store API changes.	Primary implementation.
Separate external verifier service	Strong isolation and potentially independent updates.	More IPC, lifecycle and compatibility complexity; still must integrate with QUIC/session logic.	Possible later hardening.
Compile a globally reduced root store	Simple conceptual allowlist.	Breaks ordinary browsing and private trust; does not meet selective behavior requirement.	Reject.

4.4 Data that must leave verification

The current result exposes the verified certificate and public-key hashes, plus indicators such as whether the chain is issued by a known root. The extension should make the trust decision explicit rather than infer it later. The result must identify the final anchor SPKI, the anchor source, whether the anchor was locally trusted, the matched policy rule and policy generation. These fields are security-relevant inputs to QUIC, connection pooling, diagnostics and update invalidation.

ILLUSTRATIVE RESULT EXTENSION

```
enum class TrustAnchorSource {
  kUnknown,
  kChromeRootStore,
  kPlatformUser,
  kEnterprisePolicy,
  kBrowserManagedPolicyBundle,
  kTest,
};

struct DomainTrustDisposition {
  bool rule_matched = false;
  std::string rule_id;
  uint64_t policy_generation = 0;
  SHA256HashValue trust_anchor_spki_sha256;
  TrustAnchorSource trust_anchor_source = TrustAnchorSource::kUnknown;
  bool locally_trusted_anchor = false;
  bool required_anchor_satisfied = false;
};
```

5. DomainTrustPolicy configuration extension

The browser needs a single atomic configuration object that can express both trust material and hostname-scoped requirements. The policy is intentionally separate from CACertificatesWithConstraints. Existing constrained-CA policy answers “where may this CA be used?”; DomainTrustPolicy additionally answers “which CA must be used here?” The two controls are complementary, and the private CA should still contain or receive restrictive Name Constraints.

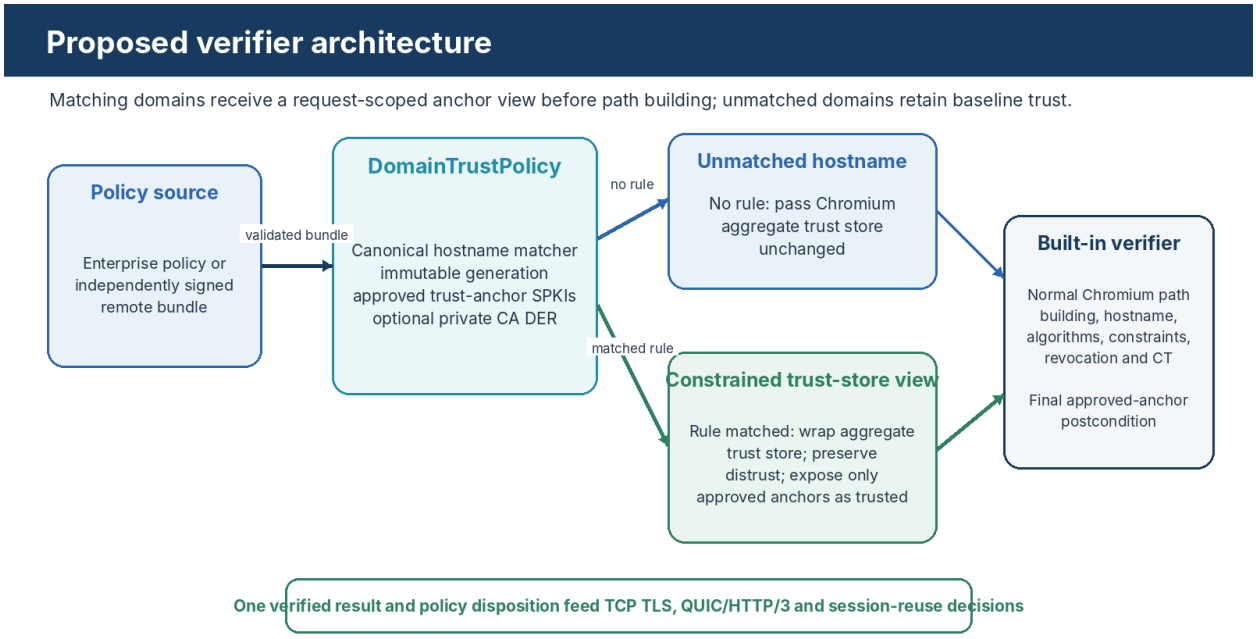


Figure 5 — Proposed browser architecture and policy flow.

5.1 Enterprise bootstrap policy

A small mandatory machine policy establishes the immutable bootstrap: whether the feature is enabled, where the independently signed bundle is obtained, which offline policy-signing keys are trusted, maximum age, emergency behavior and whether locally configured test policy is permitted. The bootstrap should change rarely and should be delivered through the same mechanisms used to enforce browser update and extension policy.

ILLUSTRATIVE MANDATORY MACHINE POLICY

```
{
  "DomainTrustPolicyEnabled": true,
  "DomainTrustPolicyUpdateUrl":
    "https://browser-policy.example.net/v1/domain-trust/component",
  "DomainTrustPolicyRootKeys": [
    "ed25519:BASE64_OFFLINE_ROOT_KEY_2026",
    "ed25519:BASE64_OFFLINE_ROOT_KEY_2027"
  ],
  "DomainTrustPolicyMaxStalenessSeconds": 604800,
  "DomainTrustPolicyExpiredMatchedDomainAction": "block",
  "DomainTrustPolicyAllowLocalTesting": false,
  "DomainTrustPolicyDiagnosticsEnabled": true
}
```

Bootstrap rule The update URL is transport, not authority. A bundle is accepted only after its content signature, generation, validity window, schema and browser compatibility have been verified against the bootstrap root keys or an authorized delegated key.

5.2 Signed bundle schema

ILLUSTRATIVE SIGNED PAYLOAD


```
{
 "schema_version": 1,
 "policy_id": "production-domain-trust",
 "generation": 184,
 "issued_at": "2026-07-26T05:00:00Z",
 "not_before": "2026-07-26T05:00:00Z",
 "expires_at": "2026-08-09T05:00:00Z",
 "minimum_browser_version": "136.0.0.0",
 "authorities": [
  {
   "id": "scone-root-2026",
   "spki_sha256": "sha256/BASE64_CURRENT_ROOT_SPKI",
   "der_base64": "OPTIONAL_BASE64_DER_CERTIFICATE",
   "trust_source": "bundle_private_anchor",
   "permitted_dns_subtrees": ["scone.cloud"]
  },
  {
   "id": "scone-root-2027",
   "spki_sha256": "sha256/BASE64_NEXT_ROOT_SPKI",
   "der_base64": "OPTIONAL_BASE64_DER_CERTIFICATE",
   "trust_source": "bundle_private_anchor",
   "permitted_dns_subtrees": ["scone.cloud"]
  }
 ],
 "rules": [
  {
   "id": "scone-services",
   "dns_subtrees": ["scone.cloud"],
   "required_trust_anchor_ids": [
    "scone-root-2026", "scone-root-2027"
   ],
   "required_intermediate_spki_sha256": [],
   "mode": "enforce",
   "failure_mode": "fail_closed",
   "not_before": "2026-07-26T05:00:00Z",
   "not_after": "2027-01-31T23:59:59Z"
  }
 ],
 "revoked_authority_spkis": [],
 "next_policy_signing_keys": [],
 "signatures": [
  {"key_id": "offline-root-2026", "alg": "EdDSA",
   "value": "BASE64_SIGNATURE"}
 ]
}
```


5.3 Field semantics

Table 10 — DomainTrustPolicy fields

Field	Required semantics
schema_version	Reject unsupported major versions. Unknown optional fields may be retained but must not change enforcement semantics.
generation	Strictly monotonic within policy_id. Persist the highest accepted generation in rollback-protected state.
issued_at / not_before / expires_at	Use secure wall-clock handling. Accept only within the validity interval; define bounded clock-skew tolerance.
minimum_browser_version	Prevents a bundle from silently relying on semantics absent from an older browser.
authorities[].spki_sha256	Canonical SHA-256 over SubjectPublicKeyInfo, encoded as sha256/<base64>. Duplicate SPKIs are rejected.
authorities[].der_base64	Optional trust material. Its SPKI must equal spki_sha256; certificate basic constraints and constraints are validated before use.
trust_source	bundle_private_anchor authorizes addition of the DER certificate to the browser-managed trust view; existing_trust_only only filters anchors already trusted by Chromium.
permitted_dns_subtrees	A safety constraint on a private anchor. The authority may not be referenced by rules outside these subtrees.

Field	Required semantics
rules[].dns_subtrees	Canonical DNS subtrees; “scone.cloud” covers the apex and complete-label descendants. Wildcard syntax is not needed.
required_trust_anchor_ids	Logical OR: any listed authority may terminate the path. At least one entry is required in enforce mode.
required_intermediate_spki_sha256	Optional additional condition. If non-empty, at least one listed intermediate SPKI must appear in the verified path.
mode	audit records the decision but preserves baseline acceptance; enforce applies the restriction.
failure_mode	fail_closed is required for production protected domains. fail_open is permitted only in a development build and must be conspicuous.
revoked_authority_spkis	Emergency content-level distrust applied before any authorization. It also causes affected sessions to close.
next_policy_signing_keys	Cross-certified delegated keys for planned signing-key rotation; bounded by activation time and scope.

5.4 DNS canonicalization and rule selection

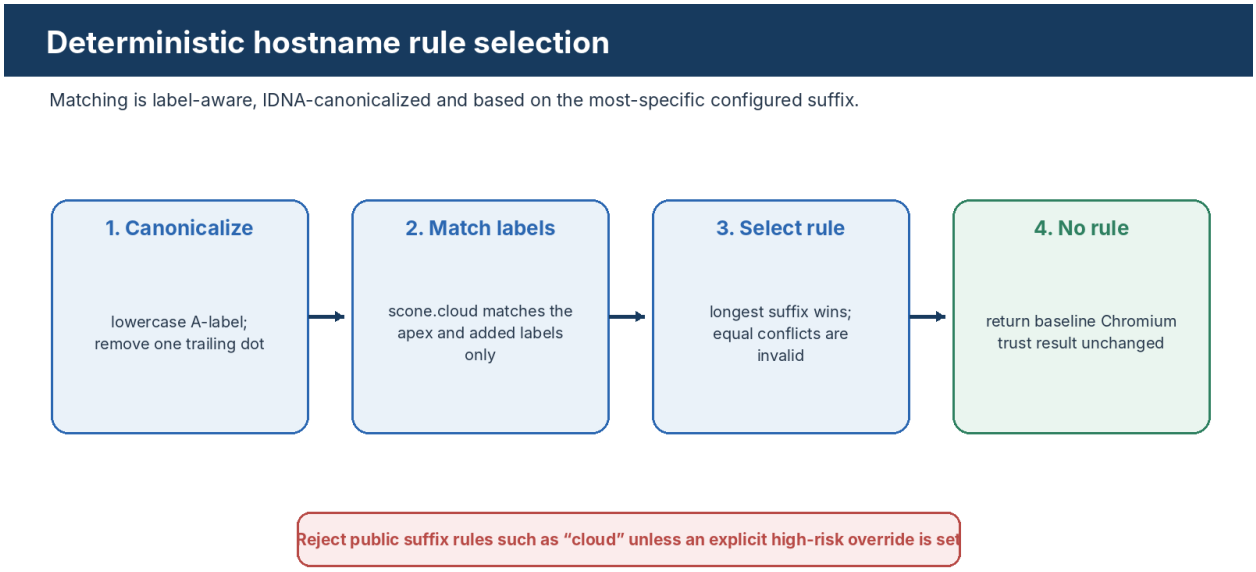


Figure 6 — DNS canonicalization and most-specific rule matching.

1. Take the reference hostname used by Chromium for certificate verification, not an untrusted Host header and not merely the displayed URL string.
2. Remove exactly one terminal dot if present; reject embedded NULs and invalid DNS syntax.
3. Convert Unicode input to the canonical IDNA A-label representation using Chromium’s hostname canonicalization rules; lowercase ASCII output.
4. Reject IP literals from DNS-subtree rules. A future schema may add explicit CIDR/IP rules with separate semantics.
5. Split into labels. A rule scone.cloud matches exactly scone.cloud or a hostname ending in .scone.cloud. It does not match evil-scone.cloud.
6. Find all matching subtrees and select the rule with the greatest number of labels. If equally specific rules conflict, reject the snapshot as ambiguous rather than relying on array order.
7. By default reject rules that are public suffixes, including cloud. Permit only organization-owned registrable domains unless an explicit high-governance override exists.

ILLUSTRATIVE COMPLETE-LABEL MATCHER

```
bool IsWithinSubtree(std::string_view host,
                     std::string_view subtree) {
  if (host == subtree) return true;
  if (host.size() <= subtree.size()) return false;
  const size_t offset = host.size() - subtree.size();
  return host[offset - 1] == '.' && host.substr(offset) == subtree;
}
```

5.5 Policy-source precedence and atomicity

The browser should expose DomainTrustPolicy as one effective snapshot. Mixing authorities from one enterprise policy source with rules from another creates an unsafe partial state. Recommended precedence is: command-line test fixture only in test builds; mandatory machine bootstrap; independently signed bundle; no user policy and no extension policy. If multiple signed bundles are supported, they must be merged by a deterministic compiler that rejects overlapping authority rather than by generic Chrome list-policy merging.

- Parse, canonicalize, validate and compile a candidate snapshot off-thread.
- Verify every referenced authority exists and every authority scope covers every referencing rule.
- Reject duplicates, ambiguous equal-specificity overlaps, malformed SPKIs and unsupported algorithms.
- Publish the compiled immutable snapshot with one atomic pointer swap only after every check succeeds.
- Keep the previous snapshot and a diagnostic when candidate activation fails.
- Expose the effective generation, signer key ID, validity and rule count in chrome://policy and a dedicated enterprise diagnostics page without exposing private DER unnecessarily.

6. Verifier algorithm and source integration

The implementation consists of a policy compiler, a request-scoped trust-store adapter, a small result extension, cache/session invalidation hooks, enterprise diagnostics and tests. The central correctness property is that unapproved roots are not eligible during path building for a matching hostname.

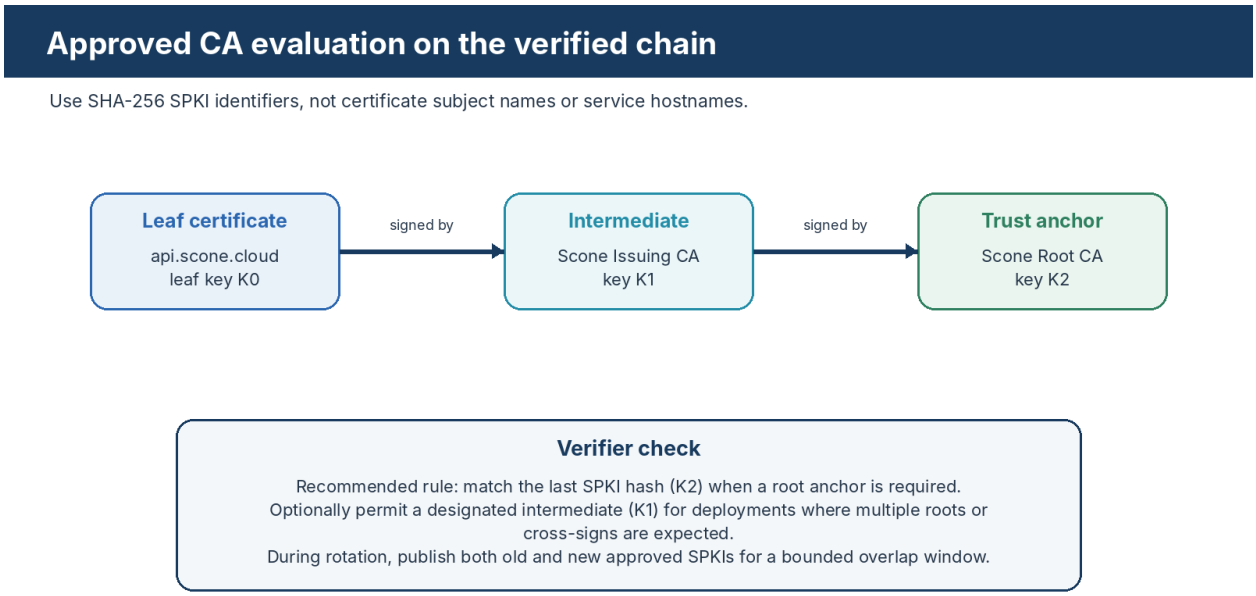


Figure 7 — Defense-in-depth evaluation of the final verified chain.

6.1 Request flow

NORMATIVE HIGH-LEVEL ALGORITHM

```
Verify(request):
  snapshot = policy_service.GetImmutableSnapshot()
  host = CanonicalizeReferenceHostname(request.hostname)
  rule = snapshot.LongestMatchingRule(host)

  if rule == none:
    return delegate.Verify(request, baseline_trust_store)

  if rule.mode == audit:
    baseline = delegate.Verify(request, baseline_trust_store)
    audit = EvaluateFinalChain(rule, baseline)
    AttachDisposition(baseline, rule, snapshot.generation, audit)
    return baseline

  allowed = snapshot.ResolveAllowedAnchorSPKIs(rule)
  filtered_store = PolicyFilteringTrustStore(
     baseline_trust_store, allowed, snapshot.revoked_spkis)
  result = delegate.Verify(request, filtered_store)

  if result != OK:
    return MapFailure(result, rule)

  if FinalAnchorSPKI(result) not in allowed:
    return ERR_CERT_REQUIRED_TRUST_ANCHOR_MISSING

  if !RequiredIntermediateSatisfied(rule, result):
    return ERR_CERT_REQUIRED_INTERMEDIATE_MISSING

  AttachSuccessfulDisposition(result, rule, snapshot.generation)
  return OK
```

6.2 PolicyFilteringTrustStore

Chromium's built-in path builder obtains issuers and trust classifications through a trust-store abstraction. The adapter delegates issuer discovery so intermediates and cross-signs remain available, but narrows positive trust-

anchor classifications for a matching request. It never promotes an unspecified certificate to trusted merely because its SPKI is listed; the approved certificate must also be trusted through the baseline store or be an authenticated private anchor supplied by the signed bundle.

ILLUSTRATIVE TRUST-STORE ADAPTER

```
class PolicyFilteringTrustStore : public bssl::TrustStore {
 public:
  void SyncGetIssuersOf(const bssl::ParsedCertificate* cert,
                        bssl::ParsedCertificateList* issuers) override {
    delegate_->SyncGetIssuersOf(cert, issuers);
  }

  bssl::CertificateTrust GetTrust(
      const bssl::ParsedCertificate* cert) override {
    auto trust = delegate_->GetTrust(cert);
    const auto spki = Sha256Spki(*cert);

    if (revoked_spkis_.contains(spki) || trust.IsDistrusted())
      return bssl::CertificateTrust::ForDistrusted();

    if (trust.IsTrustAnchor() && !allowed_anchor_spkis_.contains(spki))
      return bssl::CertificateTrust::ForUnspecified();

    return trust;
  }

  // Implement and delegate/filter additional TrustStore methods introduced
  // by the pinned BoringSSL revision, including MTC issuer methods.
};
```

Do not silently drop explicit distrust If the baseline store says a certificate or SPKI is distrusted, the filter returns distrust even when the domain rule lists that SPKI. Emergency and platform distrust must win.

6.3 Supplying browser-managed private anchors

An authority entry with trust_source=bundle_private_anchor may carry a DER certificate. The policy compiler verifies its signature-independent structure, SPKI, basicConstraints CA=true, key usage, supported key algorithm and validity policy. It also requires embedded critical Name Constraints permitting no more than the declared subtrees, or applies equivalent externally enforced constraints in the verifier. The resulting parsed certificate is added to a browser-managed trust store that is composed with the platform and Chrome Root Store before request-scoped filtering.

The distinction between authorization and trust is deliberate. A rule that lists an SPKI does not by itself introduce arbitrary certificate bytes. Either the certificate is already trusted by Chromium, or an authenticated policy authority explicitly supplies the DER and marks it as a browser-managed private anchor. This prevents a configuration typo from turning any server-presented self-signed certificate with a matching textual identifier into a root.

6.4 Result and error extensions

Table 11 — Proposed result and error changes

Element	Purpose
CertVerifyResult::domain_trust_disposition	Carries rule ID, generation, anchor SPKI/source, local/private status and enforcement outcome.
ERR_CERT_REQUIRED_TRUST_ANCHOR_MISSING	Verification could not construct a compliant path, or the defense-in-depth final check found a different anchor.
CERT_STATUS_REQUIRED_TRUST_ANCHOR_MISSING	Makes the reason available to SSL error handling, NetLog and enterprise diagnostics.
ERR_CERT_REQUIRED_INTERMEDIATE_MISSING	Optional intermediate requirement was not satisfied.
TrustAnchorSource	Distinguishes Chrome Root Store, platform/user, enterprise, browser-managed bundle and test anchors.
locally_trusted_anchor	Explicitly tells QUIC that successful verification relied on a legitimate local/private trust decision.

6.5 Where to integrate in Chromium

Exact paths must be confirmed against the pinned milestone with gn refs and code search. The following map identifies responsibilities rather than promising permanent symbol names.

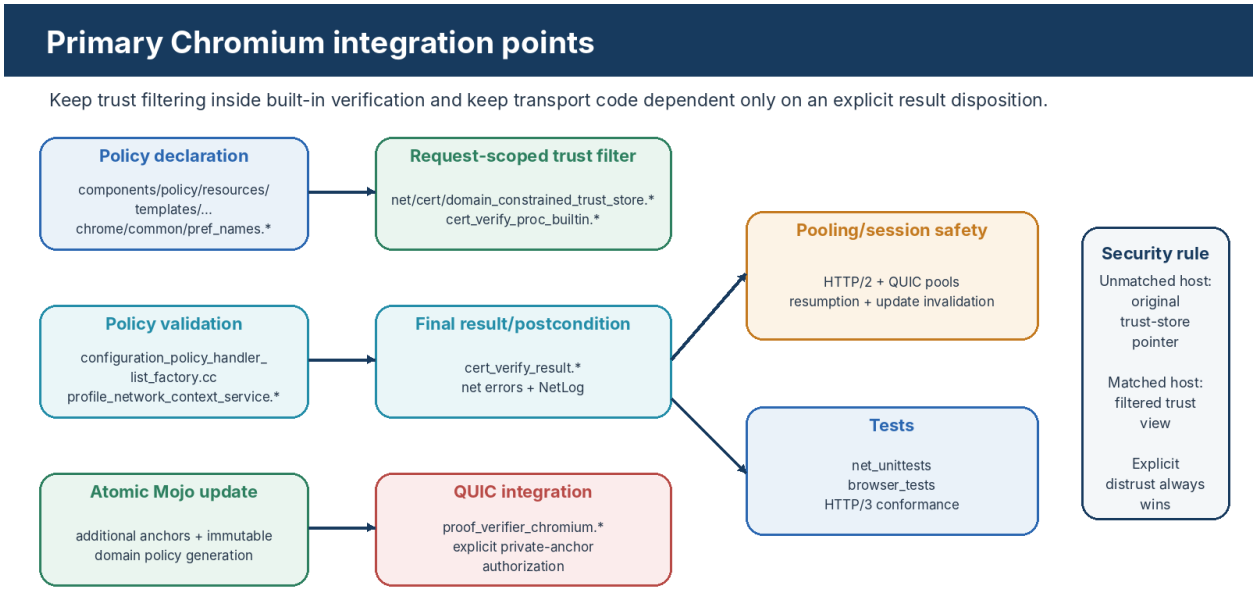


Figure 8 — Primary Chromium integration points.

Table 12 — Source integration map

Area	Representative source location	Change
Policy definition	components/policy/resources/templates/policy_definitions and generated policy schema	Define bootstrap policies, platform support, dynamic-refresh semantics and policy documentation.
Preferences / policy bridge	chrome/common/pref_names.*, policy handlers, ProfileNetworkContextService	Read mandatory bootstrap; pass effective policy configuration to the network/verifier service.
Snapshot service	New component under components/domain_trust_policy or net/cert	Verify signed bundle, compile rules, persist rollback state and publish immutable snapshots.
Mojo interface	services/cert_verifier/public/mojom or current network-service interface	Transport compiled policy generation and/or update notifications to the verifier process.
Built-in verifier	net/cert and CertVerifier implementation composition	Create request-scoped filtered trust-store view before path building; attach disposition.
Trust-store adapter	net/cert / BoringSSL pki TrustStore integration	Delegate issuers and explicit distrust; filter positive anchor trust by allowed SPKI.
Result/errors	net/cert/cert_verify_result.*, net/base/net_error_list.h, cert_status_flags.h	Add trust disposition and dedicated failure diagnostics.
TCP TLS	net/socket/ssl_client_socket_impl.cc	No duplicate policy logic; ensure disposition reaches SSLInfo and session metadata.
QUIC	net/quic/crypto/proof_verifier_chromium.*	Use local/private trust disposition in the post-verification unknown-root decision.
Caches/sessions	CachingCertVerifier, SSLClientSessionCache, HttpNetworkSession, QUIC session pool	Generation-aware cache keys and tightening-update invalidation.
Diagnostics	NetLog events, chrome://policy, certificate viewer / enterprise page	Expose matched rule and non-secret identifiers.

6.6 Prototype versus production sequence

1. Prototype a wrapper that performs only the final-chain SPKI post-check. This validates hostname matching, result propagation, errors and browser UX quickly.

2. Add alternate-path tests that demonstrate the limitation of the post-check-only prototype.
3. Implement PolicyFilteringTrustStore and move enforcement before path construction.
4. Retain the final-chain check as a CHECK-like security postcondition that returns a controlled certificate error in production.
5. Add bundle-provided private anchors and existing constraint intersection.
6. Add QUIC local/private-root disposition, then session and dynamic-update semantics.
7. Only after semantic conformance is stable, add the remote signed updater and administrative UI.

7. QUIC, HTTP/3 and locally added private trust

QUIC already uses Chromium's CertVerifier, so the domain-scoped filter applies to HTTP/3 without a second path-validation implementation. However, ProofVerifierChromium currently performs an additional check after certificate verification: when the result is successful but is_issued_by_known_root is false and the hostname is not explicitly exempted, it converts success into ERR_QUIC_CERT_ROOT_NOT_KNOWN. This protects QUIC from accepting arbitrary unknown roots, but it can also reject a legitimately locally trusted private CA. [7]

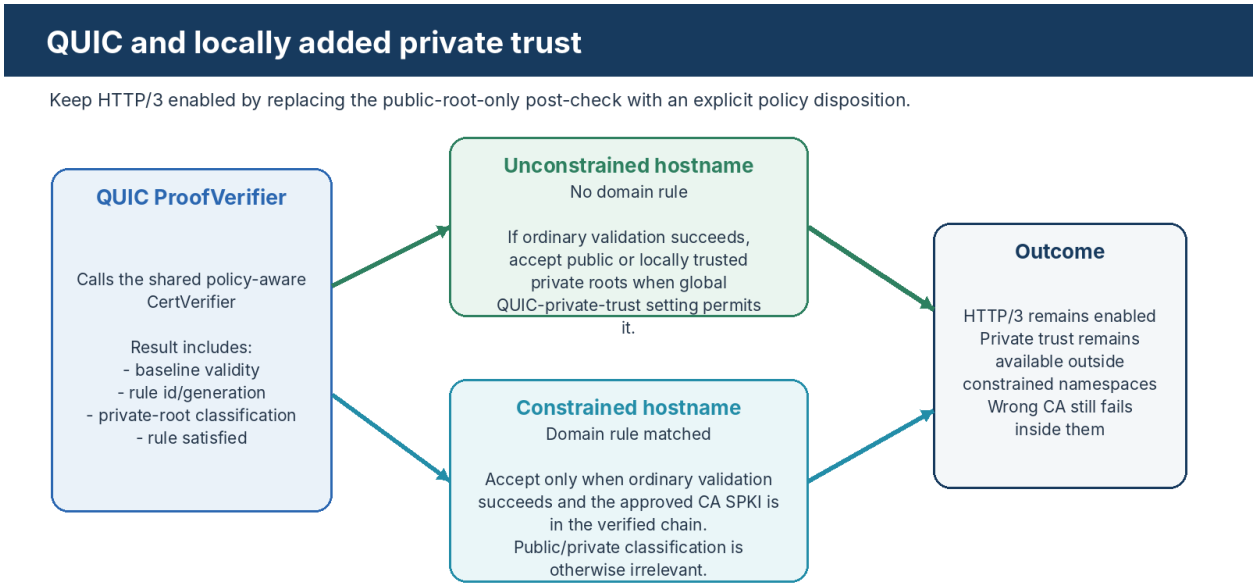


Figure 9 — QUIC behavior for public, local-private and constrained-domain trust.

7.1 Required QUIC change

ILLUSTRATIVE QUIC POST-CHECK

```
if (result == OK &&
    !verify_details_->cert_verify_result.is_issued_by_known_root &&
    !verify_details_->cert_verify_result
        .domain_trust_disposition.locally_trusted_anchor &&
    !ShouldAllowUnknownRootForHost(hostname_)) {
  result = ERR_QUIC_CERT_ROOT_NOT_KNOWN;
}
```

The locally_trusted_anchor flag is not controlled by the server. It is set only after the verifier has completed a successful path to a trust anchor whose source is a platform/user trust store, enterprise policy or authenticated browser-managed bundle. A matching domain rule is evaluated before this flag is consumed. Therefore an unapproved local root for api.scone.cloud fails during path construction or the final postcondition and never reaches QUIC as an allowed local anchor.

7.2 Behavior matrix

Table 13 — Expected TCP and QUIC outcomes

Hostname	Rule	Anchor	TCP TLS	QUIC/HTTP/3
public.example	No match	Chrome Root Store public root	Baseline success	Baseline success
intranet.example	No match	Locally installed private root	Baseline success	Success because verifier records legitimate local trust
intranet.example	No match	Self-signed server cert not in any trust store	Reject	Reject; local-trust disposition is false

Hostname	Rule	Anchor	TCP TLS	QUIC/HTTP/3
api.scone.cloud	Requires SCONE CA	Approved browser-managed private root	Success	Success
api.scone.cloud	Requires SCONE CA	Unapproved Chrome public root	Reject	Reject
api.scone.cloud	Requires SCONE CA	Unapproved local enterprise interception root	Reject	Reject
api.scone.cloud	Requires old or next SCONE CA	Next approved CA during overlap	Success	Success

7.3 Why not remove the QUIC check

Deleting the known-root check would make any root accepted by an accidentally permissive or test verifier indistinguishable from legitimate local trust. The correct change is to replace inference with an explicit trust-source result. This preserves the original defensive intent while enabling private PKI over HTTP/3.

7.4 QUIC connection pooling and origin coalescing

A QUIC connection may serve more than one origin when certificate and network rules permit coalescing. The pool must not reuse a connection across origins whose effective DomainTrustPolicy decisions differ. The session key therefore needs either the matched rule identity and policy generation or a stable policy-equivalence class. Before coalescing, Chromium must confirm that the existing verified chain satisfies the candidate origin's current rule. The same rule applies to HTTP/2 connection coalescing.

- No-rule and rule-matched origins are not automatically equivalent even when the presented certificate contains both SANs.
- Two origins governed by the same rule and generation may share only when normal Chromium coalescing requirements also pass.
- A policy update that changes either origin's equivalence class invalidates coalescing decisions.
- NetLog should record when coalescing is denied because domain-trust policy differs.

7.5 HTTP/3 fallback

The browser should preserve normal Chromium fallback behavior when QUIC fails for transport reasons. It must not reinterpret ERR_CERT_REQUIRED_TRUST_ANCHOR_MISSING as a reason to retry over TCP in the hope of obtaining a different trust result. Certificate-policy failures are transport-independent and fatal for that origin. TCP fallback is permissible only after a non-certificate QUIC failure, and the TCP verifier must use the same immutable policy snapshot semantics.

8. Connection reuse, resumption and dynamic updates

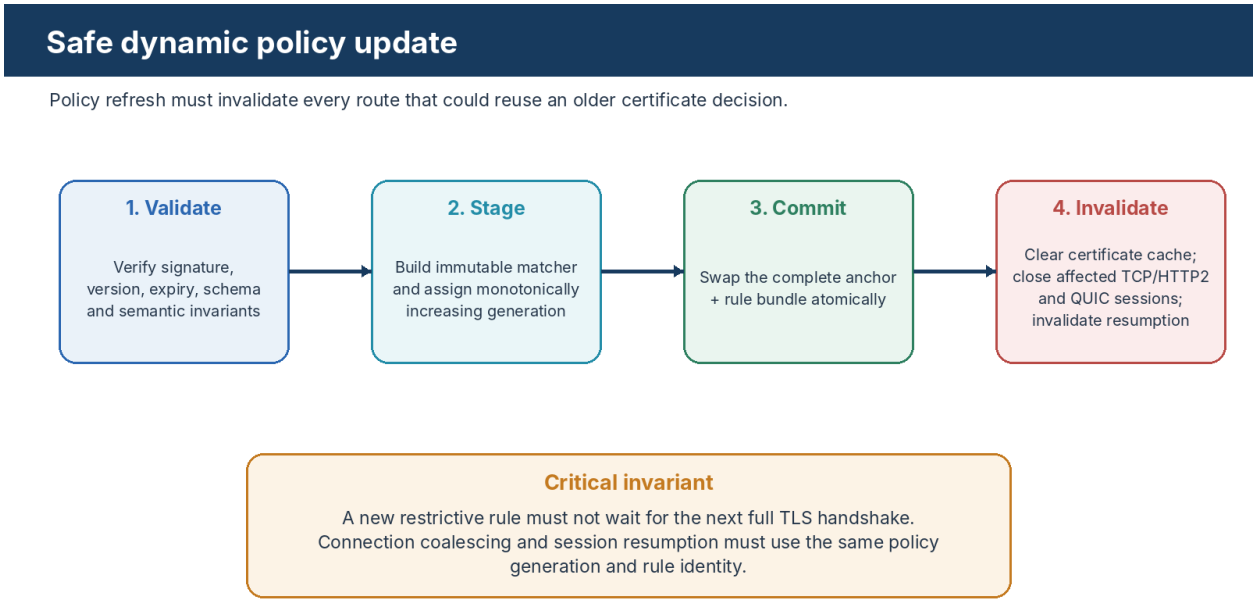


Figure 10 — Policy update, cache invalidation and connection lifecycle.

8.1 Immutable request snapshots

A verification request captures a shared immutable policy snapshot at start. An update never changes a decision half-way through path building. New requests use the new snapshot immediately after activation. The result records the captured generation so later connection and telemetry code can reason about staleness.

8.2 Verifier cache keys

CachingCertVerifier or its current equivalent must include policy semantics in cache validity. The simplest safe design clears all certificate-verifier cache entries whenever an effective policy generation changes. A later optimization may key matched entries by rule digest and preserve unaffected no-rule entries, but correctness should precede optimization.

Table 14 — Update classifications

Change	Security effect	Required action
Add a second CA to an existing rule	Loosening for planned rollover.	Clear verifier cache; new connections may use either CA. Existing compliant connections may remain until normal expiry.
Remove an old CA from a rule	Tightening.	Clear cache; close or drain TLS/HTTP2/QUIC sessions whose verified anchor is removed; invalidate resumption tickets for affected rule.
Add a new protected subtree	Tightening for that subtree.	Clear relevant caches; prevent reuse of connections established under no-rule semantics for the newly protected names.
Delete a protected subtree	Loosening.	Clear cache; existing restricted connections remain safe. New connections use baseline semantics.
Emergency revoke an SPKI	Immediate tightening and distrust.	Close every connection using that SPKI, invalidate resumption, clear cache and emit a high-severity event.
Audit to enforce	Tightening.	Treat as new protection; close noncompliant connections and prevent stale pooled reuse.

Change	Security effect	Required action
Bundle expiry	Loss of current authorization state.	For matching subtrees, block new connections and close/revalidate according to configured grace; unrelated domains remain baseline.

8.3 TLS resumption and 0-RTT

Session resumption can avoid a new certificate exchange. A resumed session must be bound to the policy-equivalence class, generation and anchor SPKI established by the original full handshake. When a tightening update affects that class or SPKI, corresponding tickets and cached sessions are invalidated. QUIC 0-RTT data must not be sent when the current policy generation cannot prove that the resumed session remains compliant. A conservative first release can disable 0-RTT for all rule-matched origins while retaining it for unmatched origins.

8.4 Existing live connections

A browser cannot retroactively change the cryptographic peer of an established TLS connection, but it can stop using the connection. The policy service should notify network contexts of tightening changes with the affected subtree and SPKI set. The HTTP and QUIC session pools then mark affected sessions non-reusable and close them promptly. Loosening changes need not terminate connections that were already compliant.

8.5 ECH, proxies and reference identity

Encrypted ClientHello does not change the identity that certificate verification authenticates. The rule matcher must use the reference hostname supplied to CertVerifier after URL and proxy processing, not plaintext SNI observed on the wire. For an explicit HTTPS proxy that terminates TLS, the browser verifies the proxy-facing certificate; exclusive upstream service validation cannot be claimed unless the proxy itself implements and attests to the same rule. For a CONNECT tunnel, ordinary end-to-end verification applies after tunnel establishment.

8.6 Failure UX and diagnostics

- Use a dedicated interstitial message: “The certificate is valid, but it is not anchored in an authority approved for this managed domain.”
- Do not offer a user bypass for mandatory enforce-mode rules.
- Show rule ID, policy generation, non-secret anchor fingerprint and a support correlation ID; do not expose full private policy bundles to untrusted pages.
- Emit structured NetLog events for rule match, allowed anchors, path result, final anchor, QUIC local-root disposition, session invalidation and update activation.
- Expose policy health to enterprise telemetry: last successful fetch, signature key, generation, expiry, update error and count of blocked connections.

9. Secure centralized configuration

Central management is required because the security rule must change during CA rotation and as new protected namespaces are onboarded. The browser must not equate “downloaded over HTTPS” with “authorized trust policy.” A compromised web server, CDN, TLS CA or network path must not be able to introduce a new browser trust anchor. The recommended design uses a small machine-policy bootstrap and independently signed, versioned policy bundles distributed through Chromium’s component-update machinery or an equivalent organization-controlled updater.

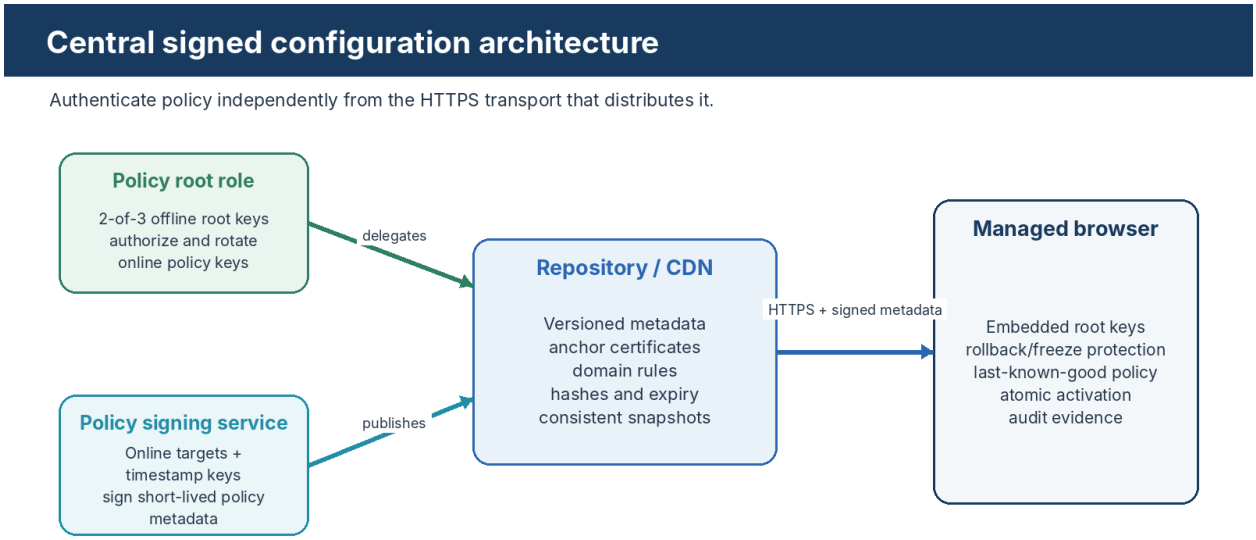


Figure 11 — Signed centralized configuration and trust separation.

9.1 Trust hierarchy

Table 15 — Distinct keys and responsibilities

Key / authority	May authorize	Must not authorize
Browser binary signing key	Installation and update of the browser executable.	Domain rules or TLS certificates by itself.
Enterprise bootstrap policy	Feature enablement, policy update endpoint and offline policy root keys.	Dynamic CA changes without a valid content signature.
Offline domain-policy root	Policy signing-key delegation, emergency revocation, root-key rotation and recovery.	Routine online TLS issuance.
Online policy signing key	Versioned policy bundles within its environment and time/scope delegation.	Browser binaries or arbitrary code.
Namespace delegation key	Rules and authorities only below an assigned DNS subtree.	Sibling or parent namespaces.
TLS CA key	Server certificates within its Name Constraints and issuance policy.	Browser policy changes unless separately enrolled as a narrowly scoped rollover delegate.

9.2 Distribution mechanisms

Table 16 — Distribution options

Mechanism	Advantages	Risks / controls	Recommendation

Mechanism	Advantages	Risks / controls	Recommendation
Private CRX/component updater	Reuses Chromium scheduling, staged rollout and component lifecycle.	Requires private update service and careful component-key ownership; payload still needs independent content signature.	Preferred when maintaining a browser fork.
Dedicated HTTPS fetcher in browser	Simple protocol and full control.	Creates new updater code, proxy behavior, retry logic and attack surface.	Acceptable only with strong tests and signed content.
Static mandatory policy bundle	No runtime updater; easy for air-gapped endpoints.	Slow rotation and domain onboarding; MDM/GPO policy-size constraints.	Use for offline profiles or emergency recovery.
Browser extension	Easy distribution.	Wrong trust boundary and lifecycle; extension compromise could change policy.	Do not use as authority.

9.3 Signed envelope and canonicalization

Use a deterministic encoding so every verifier signs identical bytes. Canonical CBOR with COSE signatures is a strong choice; canonical JSON with a precisely specified JWS profile is also viable. The signature covers every semantic field, including generation, validity, authority DER, rule mode, delegations and next keys. Reject duplicate map keys and non-canonical encodings before semantic parsing.

- Require at least one currently authorized signature; optionally require threshold signatures for production or emergency policy.
- Bind signatures to policy_id and deployment environment to prevent a staging bundle from being accepted in production.
- Verify the complete chain of policy-key delegation and its DNS/time scope.
- Persist the highest accepted generation and signing-key epoch in a tamper-resistant or at least integrity-protected local state file.
- Write the new bundle to a temporary file, fsync, verify by rereading, then atomically rename. Never update separate authority and rule files independently.
- Retain the previous two valid snapshots for diagnostic rollback, but do not permit rollback below the persisted monotonic generation without an offline recovery authorization.

9.4 Availability and expiry policy

The browser should continue using the last-known-good snapshot during a temporary outage. The bundle validity window must be long enough for realistic endpoint sleep and travel, while the update interval remains short enough for emergency action. A typical policy could refresh every 6 hours, alert after 48 hours without success and expire after 7–14 days. After expiry, only matching protected domains fail closed; unmatched domains continue to use baseline trust. This limits collateral outage without weakening the security claim for protected services.

9.5 Policy signing-key rotation

1. Offline root signs a delegation containing both current and next online signing keys, activation times, expiry and environment scope.
2. Clients receive and persist the delegation while bundles are still signed only by the current key.
3. Controller begins dual-signing policy bundles. Telemetry confirms that the next key is recognized across the fleet.
4. Controller switches to the next key only. The current key remains authorized for a short rollback window.
5. Offline root publishes a later delegation that removes the old key. Emergency compromise uses an immediate root-signed revocation and generation jump.

10. Automated CA rotation and domain onboarding

10.1 CA rotation principles

CA rotation must be automated but governed. The original CA certificate can help authenticate a rollover request, but it should not be the sole permanent policy authority: a compromised CA could otherwise authorize its own malicious successor. The controller accepts a rollover object signed by the current CA or a registered rollover key, validates scope and key possession, and requires the configured organizational approval. The offline domain-policy hierarchy remains the final browser-policy authority.

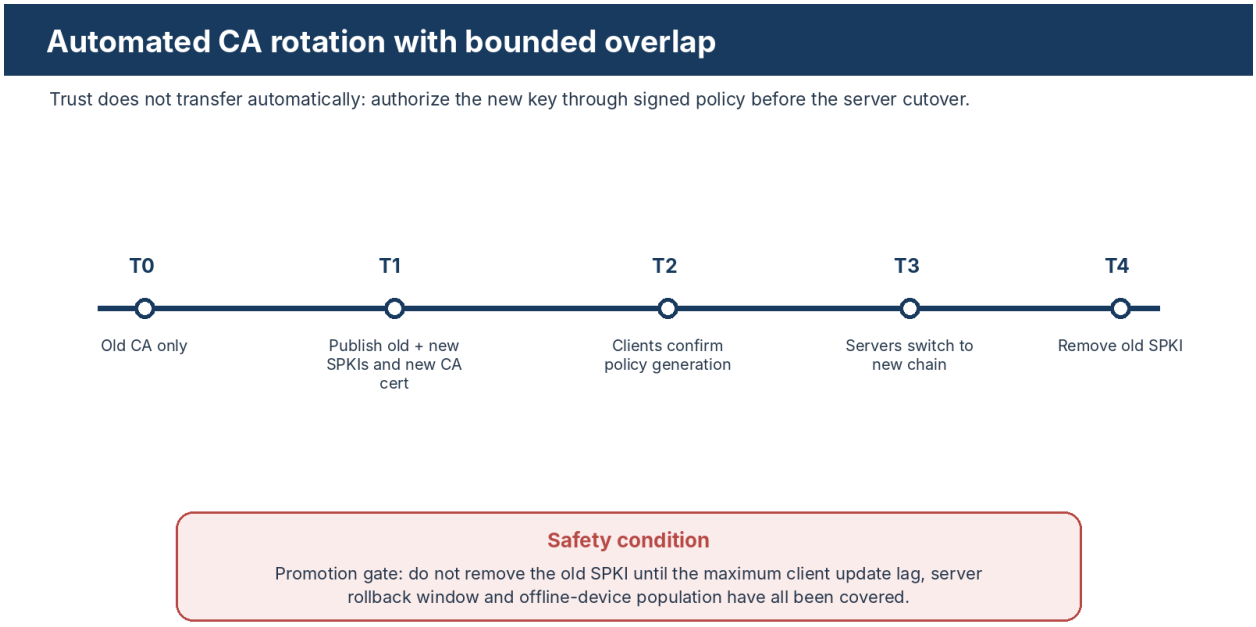


Figure 12 — Automated CA rotation with overlap and removal.

10.2 Rotation object

ILLUSTRATIVE ROLLOVER REQUEST

```
{
 "operation": "rotate_authority",
 "namespace": "scone.cloud",
 "current_authority_spki": "sha256/OLD",
 "next_authority": {
  "spki_sha256": "sha256/NEW",
  "der_base64": "BASE64_DER_NEW_CA",
  "permitted_dns_subtrees": ["scone.cloud"]
 },
 "activation_not_before": "2026-09-01T00:00:00Z",
 "old_authority_remove_after": "2026-10-15T00:00:00Z",
 "nonce": "CONTROLLER_CHALLENGE",
 "proofs": [
  {"type": "current_ca_signature", "value": "..."},
  {"type": "next_key_possession", "value": "..."}
 ]
}
```

10.3 Automated rollover runbook

1. Controller issues a nonce and receives a request signed by the registered current-CA rollover identity. The next CA proves possession of its private key.
2. Validate that the next certificate is a CA, uses an accepted algorithm, is not explicitly distrusted, and carries critical embedded Name Constraints equal to or narrower than the delegated namespace. Calculate certificate and SPKI fingerprints independently.
3. Run policy linting, public-suffix/overlap checks and governance approval. Generate a candidate policy with both old and next authority IDs in the rule.

4. Publish the overlap bundle to the audit/pilot channel. Test TCP, HTTP/3, cross-signs, alternate AIA paths, revocation and browser restart. Confirm fleet adoption telemetry.
5. Promote overlap to production. Only after a threshold of clients reports the new generation may service issuance and server chains switch to the next CA.
6. Observe the service for the defined soak period. Reject any server still using an unrelated CA throughout the transition.
7. Publish a tightening bundle removing the old SPKI. The browser invalidates sessions and resumption state anchored at the old CA.
8. Revoke or offline the old CA according to PKI policy and retain the signed audit record.

Same-key reissuance When a CA certificate is renewed with the same SubjectPublicKeyInfo and compatible constraints, the domain rule's SPKI allowlist need not change. The new DER may still need distribution when the browser-managed bundle supplies the anchor certificate.

10.4 Emergency CA replacement

If the current CA key may be compromised, do not require its approval. The offline policy root or a threshold emergency key publishes a generation jump that places the old SPKI in revoked_authority_spkis, adds the emergency replacement CA and shortens the bundle refresh interval. Clients close affected sessions immediately. Service operators then deploy certificates from the replacement CA. The old CA's signature is evidence, not authority, in this path.

10.5 Automated domain onboarding

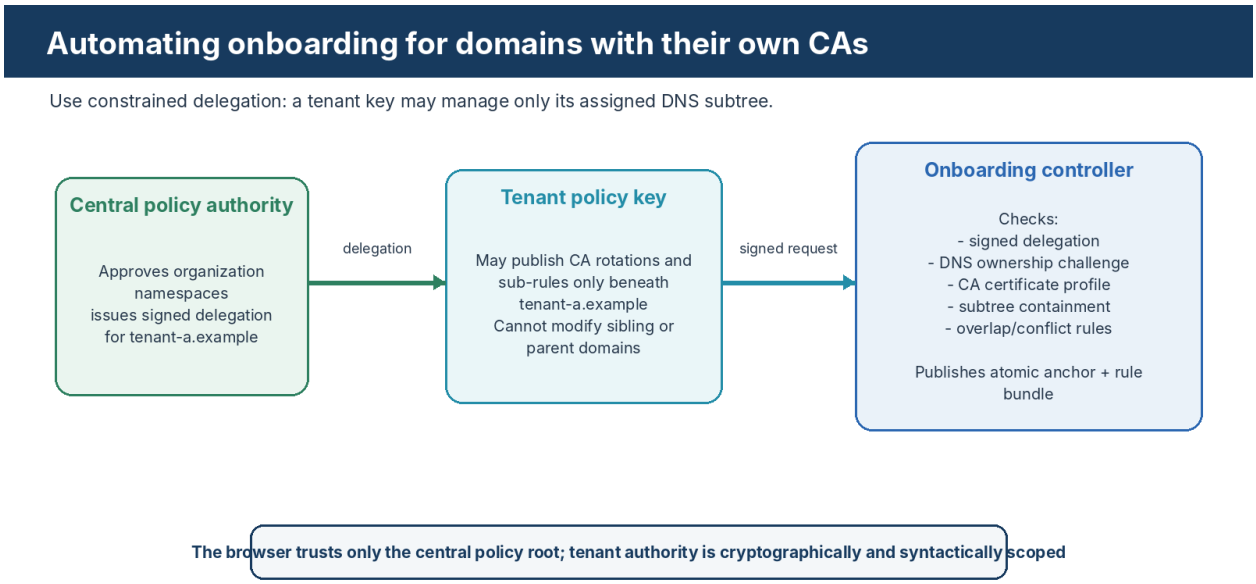


Figure 13 — Governed onboarding of a new namespace and its CA.

1. Requester submits the exact DNS subtree, CA DER/SPKI, desired activation date, operational contacts and whether the CA is existing-trust-only or browser-managed private trust.
2. Controller proves DNS control with a nonce-bearing TXT record under a designated onboarding label. DNSSEC is preferred but not the sole control; organizational identity and repository approval remain required.
3. CA proves private-key possession by signing a controller nonce. The controller validates basic constraints, key usage, algorithms, validity and critical Name Constraints.
4. Reject public suffixes, sibling-domain claims and overlaps with an existing delegation unless an explicit parent authority approves. Longest-match semantics are not a substitute for governance.
5. Generate test endpoints: approved chain, unapproved public chain, unapproved local-private chain, wrong SAN, cross-signed alternate path and HTTP/3.

6. Publish the new rule in audit mode. Collect telemetry showing whether existing services use a non-approved CA or a chain not compatible with the future rule.
7. After remediation and approval, publish enforce mode with a future not_before time. Confirm clients have received the policy before changing service assumptions.
8. Continuously monitor certificate inventory and CT logs where applicable; initiate governed rotation before the CA or signing delegation expires.

10.6 Delegated namespace management

Large deployments should not centralize every routine change in one key. The offline policy root can issue a delegation to a team key for dev.scone.cloud. The delegation contains permitted DNS subtrees, permitted authority SPKIs or rules for introducing new SPKIs, maximum validity, maximum overlap period, environment and a monotonic sub-generation. The controller verifies that delegated output cannot escape its subtree. A rule for app.dev.scone.cloud may be signed by that delegate; a rule for scone.cloud or prod.scone.cloud may not.

ILLUSTRATIVE CONSTRAINED DELEGATION

```
{
  "delegate_key_id": "team-dev-2026",
  "public_key": "ed25519:BASE64",
  "permitted_dns_subtrees": ["dev.scone.cloud"],
  "may_add_private_authorities": true,
  "maximum_rule_validity_seconds": 7776000,
  "maximum_ca_overlap_seconds": 2678400,
  "not_before": "2026-08-01T00:00:00Z",
  "expires_at": "2027-08-01T00:00:00Z",
  "signature_by": "offline-domain-policy-root"
}
```

11. Build and development runbooks

11.1 Development workstation and capacity planning

Chromium builds are large and toolchain-sensitive. Use dedicated builders rather than relying on developer laptops for release artifacts. Keep at least several hundred gigabytes of fast local SSD space per active source/output tree, substantial RAM and a recent multi-core CPU. The exact dependencies, supported host distribution and Xcode/SDK versions must be taken from the pinned Chromium milestone's official Linux and macOS build instructions. [3][4]

11.2 Fast upstream-Chromium prototype on Linux

REPRESENTATIVE UPSTREAM PROTOTYPE FLOW

```
# Install and prepend depot_tools to PATH, then:
mkdir chromium && cd chromium
fetch --nohooks chromium
cd src
gclient runhooks

gn gen out/DomainTrust --args='
  is_debug=true
  is_component_build=true
  symbol_level=1
  enable_nacl=false
'

autoninja -C out/DomainTrust chrome net_unittests unit_tests browser_tests
out/DomainTrust/chrome --user-data-dir=/tmp/domain-trust-profile
```

Use this direct checkout to locate current classes, build small test targets and validate the design before packaging. `gn refs out/DomainTrust <source-file>` and `gn desc` should be used to discover current target labels rather than hard-coding labels from an older milestone. Official Chromium Linux instructions remain the source of truth for prerequisites and depot_tools usage. [3]

11.3 Private ungoogled-chromium portable-Linux build

PORTABLE-LINUX PRODUCT BUILD

```
git clone --recurse-submodules \
  ssh://git.example.net/browser/browser-portablelinux.git
cd browser-portablelinux
git checkout --recurse-submodules release/<milestone>-scone.1

# Build from a Chromium source clone rather than a release tarball:
scripts/docker-build.sh -c

# Produce AppImage and tar.xz artifacts:
scripts/package.sh
# or: package/docker-package.sh

ls -lh build/
```

The upstream portable-Linux repository documents scripts/docker-build.sh as the supported containerized build entry point, scripts/package.sh or package/docker-package.sh for packaging, and `-c` for development builds from a source-tree clone. [5] The private fork should pin the container base image by digest and publish the builder image separately with an SBOM.

11.4 Linux developer patch loop

REPRESENTATIVE PATCH-DEVELOPMENT LOOP

```
# In a prepared build/src tree with QUILT_PATCHES pointing at the merged series:
quilt push -a
# Edit Chromium source or start a patch:
quilt new secure-domain-trust/0003-request-scoped-trust-store-filter.patch
quilt add net/cert/<current-files>
$EDITOR net/cert/<current-files>
quilt refresh

# Validate and build focused targets:
../../ungoogled-chromium/devutils/validate_config.py
../../ungoogled-chromium/devutils/validate_patches.py \
  -l . -s ../../patches/series.merged
gn refs out/DomainTrust net/cert/<changed-file>
autoninja -C out/DomainTrust net_unittests unit_tests
```

The exact helper paths depend on the packaging repository. Preserve the core rule: each patch applies with no fuzz, validation runs before commit, and focused tests run before a full browser build.

11.5 Private ungoogled-chromium macOS build

MACOS PRODUCT BUILD

```
git clone --recurse-submodules \
  ssh://git.example.net/browser/browser-macos.git
cd browser-macos
git checkout --recurse-submodules release/<milestone>-scone.1

# Native architecture (arm64 on Apple Silicon, x86_64 on Intel):
./build.sh

# x86_64 build on Apple Silicon when supported by the pinned scripts:
./build.sh x86_64

ls -lh build/*.dmg
```

The upstream macOS packaging repository documents `./build.sh` for the host architecture and `./build.sh x86_64` for an Intel build on Apple Silicon, with a DMG produced under build/. [6] Production releases must use organization-controlled signing and notarization identities. The packaging repository shows a codesign/notarytool flow, but the exact entitlements and required Xcode/SDK version must be revalidated for the pinned Chromium milestone and the private browser identity. [6]

11.6 macOS patch refresh and rebase

REPRESENTATIVE MACOS PATCH-MAINTENANCE FLOW

```
./devutils/update_patches.sh merge
source devutils/set_quilt_vars.sh
mkdir -p build/{src,download_cache}
# Retrieve/unpack resources for the selected architecture using repo scripts.
cd build/src
quilt push -a --refresh
# On failure:
quilt push -f
quilt edit <affected-file>
quilt refresh
# Repeat until all patches apply.
quilt pop -a
cd ../..
./ungoogled-chromium/devutils/validate_config.py
./ungoogled-chromium/devutils/validate_patches.py \
  -l build/src -s patches/series.merged
./devutils/update_patches.sh unmerge
./devutils/check_patch_files.sh
```

This follows the downstream macOS repository's documented merge, Quilt refresh, validation and unmerge sequence. [6] Resolve semantic conflicts manually and rerun the security tests; a refreshed patch that applies is not necessarily a correct verifier.

11.7 macOS signing, notarization and updates

- Use a dedicated Developer ID Application certificate and a separate installer/update signing identity where applicable.
- Apply hardened runtime and only the entitlements required by the Chromium milestone and product features. Verify nested code signatures before notarization.
- Submit the final app archive with notarytool, wait for success, staple the ticket and run Gatekeeper assessment on a clean test machine.
- Serve updates through an organization-controlled signed update channel. Test rollback protection and code-signing identity rotation independently of domain-policy signing.
- Maintain arm64 and x86-64 artifacts only as long as required. A universal binary can simplify distribution but increases build and test scope.

11.8 Debugging and inspection tools

Table 17 — Developer diagnostics

Tool	Use
chrome://policy	Verify mandatory bootstrap values, source, scope, status and refresh.

Tool	Use
Enterprise domain-trust page	Show effective generation, signer, expiry, rule IDs and authority SPKIs.
chrome://net-export / NetLog viewer	Capture path result, rule match, trust source, QUIC decision and connection reuse.
chrome://histograms	Inspect update, match and enforcement metrics.
openssl s_client / quiche or test server	Inspect served chains and create controlled TCP/HTTP3 cases; browser behavior remains authoritative.
gn refs / gn desc	Find current build targets and dependency ownership after a Chromium rebase.
lldb / gdb	Break in policy matching, trust-store GetTrust, result attachment and QUIC post-check.

12. Test strategy and conformance laboratory

The feature changes a browser security boundary. Tests must prove not only that protected domains are restricted, but also that unrelated domains retain baseline Chromium trust—including locally installed private roots and QUIC. A layered test program combines pure unit tests, verifier/path-building tests, QUIC tests, browser integration tests, platform trust tests, signed-update lifecycle tests and release-channel canaries.

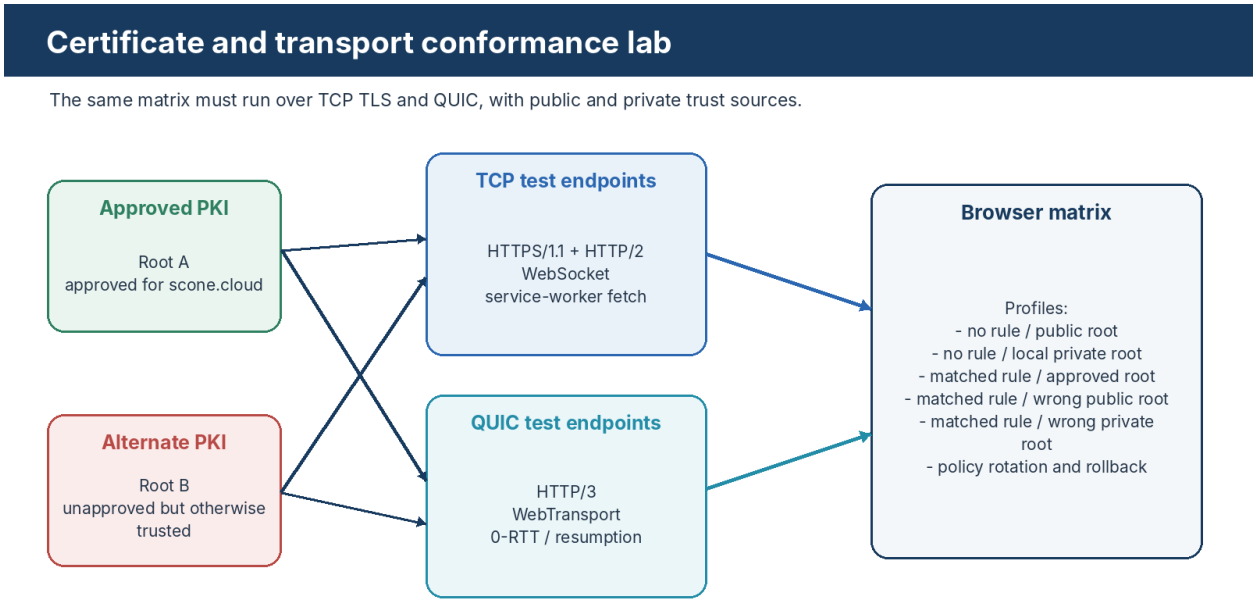


Figure 14 — Multi-protocol conformance laboratory.

12.1 Test PKI

Table 18 — Required deterministic test authorities

Identifier	Purpose
R-approved-old	Current private approved root for scone.cloud.
R-approved-next	Next approved root used for rollover overlap.
R-public	Representative ordinary trusted/public root in the test verifier.
R-local	Locally installed private root used only for an unmatched intranet domain.
R-intercept	Enterprise interception root that is trusted generally but unapproved for scone.cloud.
R-revoked	Previously approved SPKI placed in emergency policy revocation.
I-cross-A/B	Cross-signed intermediates that can build to approved and unapproved roots.
Leaf sets	Apex, multi-level, IDN A-label, wildcard, multi-SAN, wrong-name, expired, weak-algorithm and revoked leaves.

12.2 Unit tests

- Canonicalization: case, trailing dot, Unicode/U-label to A-label, invalid names, IP literals and embedded delimiters.
- Matching: apex, one/multiple labels, evil-scone.cloud suffix confusion, most-specific match and equal-specificity conflict rejection.
- Schema: duplicate keys, duplicate SPKIs, dangling authority IDs, scope escape, unsupported mode, invalid time windows and public-suffix rejection.
- Signature and rollback: wrong key, expired delegation, generation replay, environment confusion, threshold failure and valid next-key activation.

- Trust filter: issuer delegation, explicit distrust precedence, unapproved anchor becomes unspecified, approved baseline anchor remains trusted, approved bundle anchor becomes trusted only after DER verification.
- Result disposition: source classification, final anchor SPKI, local-private flag and matched generation.

12.3 Path-building tests

Use Chromium's certificate test utilities and built-in verifier test harness. Include a case where the baseline path builder selects an unapproved path while an approved path exists; prove that the post-check-only prototype fails and the pre-path trust-store filter succeeds. Include AIA-fetched and cached intermediates, cross-signed roots, path-length constraints, embedded Name Constraints and an explicitly distrusted approved SPKI.

12.4 QUIC tests

- Unmatched intranet hostname plus R-local: TCP and QUIC both succeed.
- Unmatched hostname plus unknown self-signed root: TCP and QUIC both reject.
- Matched api.scone.cloud plus R-approved-old: TCP and QUIC succeed.
- Matched api.scone.cloud plus R-public or R-intercept: both transports reject with the required-anchor error, not a generic QUIC-only root error.
- Matched domain during old/new overlap: either approved root succeeds; after removal, old-root QUIC sessions close and new handshakes to old fail.
- HTTP/3 fallback after a policy failure does not silently succeed over TCP.
- QUIC connection coalescing does not cross different rule IDs or generations.

12.5 Browser-level integration tests

Table 19 — Browser surfaces that must share enforcement

Surface	Required cases
Top-level navigation	Positive approved path, unapproved public path, interstitial without bypass.
Fetch/XHR	Same rule and error as navigation; no CORS-dependent bypass.
WebSocket	Initial handshake and automatic reconnect after CA/policy change.
Service worker	Install, update and background fetch; stale worker does not bypass new policy.
Preconnect / speculative networking	No connection is retained when its eventual origin requires a different rule.
HTTP/2 and HTTP/3 coalescing	SAN overlap plus same/different policy equivalence classes.
Redirects	Each new origin is evaluated independently; redirect from unmatched to matched cannot reuse an ineligible connection.
ECH	Rule uses reference identity and remains correct with encrypted ClientHello.
Proxy modes	Direct, CONNECT tunnel and terminating proxy behavior clearly distinguished.
Incognito / profiles	Mandatory machine policy applies consistently; snapshot service does not leak user data between profiles.

12.6 Platform tests

On Linux, test the exact trust integration used by the chosen build and desktop environment, including user-added and enterprise anchors where supported. On macOS, create temporary keychains or managed profiles for R-local and R-intercept, confirm baseline Chromium sees them, then validate matched and unmatched behavior. Clean up trust material after each test and run on disposable VMs or snapshots.

12.7 Update and lifecycle tests

- Atomic activation while many parallel handshakes are running: each request uses one complete generation.
- Network outage and last-known-good use; matched domain fails only after configured expiry while unmatched browsing remains normal.
- Generation rollback and stale mirror: candidate rejected, current state preserved.
- Audit to enforce transition with existing noncompliant HTTP/2 and QUIC sessions.
- Old+new CA overlap, server cutover, old removal, emergency revocation and browser restart at every stage.
- Policy signing-key dual-sign and rotation; namespace delegation expiry and attempted scope escape.

- Binary/browser update with policy schema migration and a bundle requiring the new minimum version.

12.8 Fuzzing and negative testing

- Fuzz canonical hostname and rule matching with Unicode, extremely long labels, unusual dots, malformed A-labels and parser differentials.
- Fuzz signed-bundle parsing, canonicalization and duplicate-key handling independently of signature verification.
- Fuzz authority DER and constraint intersection; never use certificate subject/issuer strings as authority identity.
- Run differential tests between baseline Chromium and the fork for a large corpus of unmatched domains and certificate chains.
- Use fault injection at policy activation, persistent-state writes, cache clearing and connection-closing boundaries.
- Include mutation tests that deliberately invert allow/deny branches to prove the conformance suite detects a fail-open implementation.

13. CI, release engineering and operations

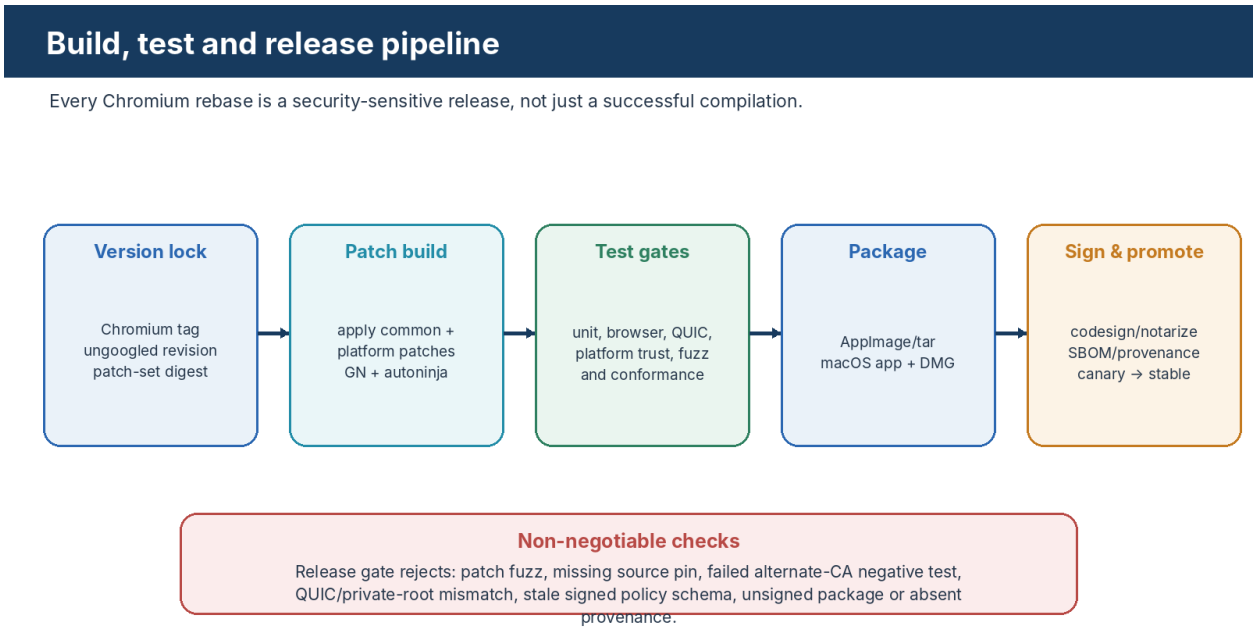


Figure 15 — Source-to-release pipeline with security gates.

13.1 Pipeline stages

1. Source intake: verify Chromium and downstream revisions, source archives, submodules and toolchain digests.
2. Patch validation: merge platform series, apply private patches with zero fuzz, run format/static analysis and verify generated files are current.
3. Focused build: compile net/cert, QUIC, policy and updater test targets; run unit/path tests.
4. Full build: produce browser binaries for Linux and macOS architectures from clean workers.
5. Conformance lab: launch deterministic TCP/HTTP2/HTTP3 endpoints and run the complete positive/negative matrix.
6. Platform qualification: install packages on clean supported OS images, add/remove local roots, exercise enterprise policy and updates.
7. Artifact assurance: generate SBOM, vulnerability scan, license bundle, symbols and signed provenance.
8. Signing/notarization: use isolated release jobs and hardware-backed identities.
9. Promotion: development to internal pilot, controlled external pilot and stable only after telemetry and rollback readiness are approved.

13.2 Release cadence and Chromium security updates

A private browser inherits the obligation to consume Chromium security releases quickly. The patch group must therefore remain small and layered. Establish an emergency path that imports a security milestone, rebases the security patches, runs a shortened but mandatory critical conformance set, builds and signs packages, then follows with the full regression suite. If the verifier patch cannot be rebased safely, block access to protected services or retain the last secure browser only for the minimum bounded interval; do not ship an untested partial merge.

13.3 Channels and policy compatibility

Table 20 — Recommended release channels

Channel	Population	Policy behavior
dev	Developers and CI	May allow local test bundles and verbose diagnostics; never used for production credentials.

Channel	Population	Policy behavior
pilot	Managed representative endpoints	Production signature roots; audit or enforce rules with enhanced telemetry.
stable	Production fleet	Mandatory signed bundles, fail-closed matching domains, normal diagnostics.
recovery	Restricted break-glass package	Known-good browser plus pinned recovery policy root; distributed only under incident procedure.

13.4 Fleet telemetry

- Browser version, patch revision and platform—not full browsing history.
- Effective policy ID/generation, signer key ID, expiry and last successful update.
- Counts of rule matches, approved successes, required-anchor failures, malformed-policy attempts and emergency revocations.
- Anchor SPKI identifiers only for protected-domain decisions, with retention and access controls appropriate to security telemetry.
- QUIC versus TCP outcomes, local-private-root acceptance on unmatched domains, and policy-caused connection invalidation.
- No certificate private keys, page content, cookies, passkey data or full private bundle material.

13.5 Operational SLOs

Table 21 — Suggested operational objectives

Metric	Objective
Chromium critical security update integration	Organization-defined maximum; measure from authoritative release to stable fork deployment.
Policy update success	At least 99.9% of active managed clients within 24 hours, with explicit sleeping/offline classification.
CA overlap readiness	Target fleet threshold reached before service certificates move to the next CA.
Policy verification failure	Alert immediately; no invalid snapshot activation.
Protected-domain false acceptance	Zero in conformance and production incident monitoring.
Unmatched-domain behavioral regression	No statistically significant certificate-error increase versus baseline canary.

14. Security analysis and residual risk

14.1 Security properties

Table 22 — Achieved properties and conditions

Property	Achieved when
Exclusive anchor for a matching subtree	Trust-store filtering is active before path construction, the final postcondition passes and the policy snapshot is valid.
No collateral restriction on unmatched domains	No-rule path delegates unchanged to baseline trust and session/cache logic does not carry a matched restriction across origins.
Private PKI over QUIC for unmatched domains	The verifier records legitimate local trust and QUIC consumes that disposition rather than only is_issued_by_known_root.
Consistent TCP/QUIC enforcement	Both transports use the same CertVerifier snapshot and certificate-policy errors are not bypassed by fallback.
Safe CA rollover	Old/new overlap is distributed before server cutover and old sessions are invalidated after removal.
Central policy authenticity	Content signatures, key delegation, rollback state and atomic last-known-good activation all validate.

14.2 Residual risks

Table 23 — Residual risk and mitigation

Risk	Impact	Mitigation
Fork falls behind Chromium security releases	Known browser vulnerabilities outweigh the trust feature.	Small patch delta, dedicated rebase team, emergency release process and measurable update SLO.
Endpoint administrator replaces browser/bootstrap keys	Policy can be bypassed locally.	Managed device hardening, signed packages, OS integrity controls and attestation where applicable.
Policy-controller compromise	Malicious authorized bundle within compromised key scope.	Offline root, constrained online keys, threshold approvals, transparency/audit log and short validity.
CA and policy key share a failure domain	CA compromise can become policy compromise.	Separate HSM partitions, identities and governance; CA signatures are rollover evidence only.
Path-builder API changes during rebase	Filter may become incomplete or semantically wrong.	Source integration map, compile-time interface coverage, alternate-path tests and re-review every milestone.
Connection reuse bug	A stale or differently governed connection serves a protected origin.	Policy equivalence in pool keys, update invalidation and dedicated coalescing tests.
Clock manipulation	Expired or not-yet-valid bundle accepted/rejected incorrectly.	Bounded skew, monotonic elapsed-time checks after first validation, secure time sources and telemetry.
User confusion between private fork and Chrome	Support and update mistakes.	Distinct branding, application ID, update channel and enterprise documentation.

14.3 Fail-open analysis

The implementation must make fail-open paths observable and testable. A missing or expired policy cannot silently become “no rule” for a hostname that was previously protected. The browser persists a protected-subtree summary with the last-known-good snapshot; after expiry it blocks those subtrees until a valid successor or authorized recovery bundle arrives. Conversely, a policy service failure must not globally disable public browsing for names that were never protected.

14.4 Why SPKI rather than certificate fingerprint

SPKI identifies the CA public key and tolerates a controlled reissuance of the CA certificate with the same key. It also aligns with Chromium's existing public-key hash representation. The controller still records the complete certificate fingerprint for inventory and audits. If the organization requires every CA certificate reissue to be explicitly approved even with the same key, the schema may add an optional DER-certificate hash condition in addition to—not instead of—the SPKI requirement.

14.5 Privacy

Domain rules are enterprise configuration and may reveal internal namespaces. Store bundles with restrictive file permissions, redact them from general crash dumps and avoid exporting full rule lists to websites or extensions. Telemetry should aggregate outcomes and use rule IDs rather than complete URLs. Policy update requests should not enumerate the site currently being visited.

15. Implementation roadmap and conclusions

15.1 Phased roadmap

Table 24 — Suggested phases

Phase	Scope	Exit criteria
0 — Architecture spike	Direct upstream Chromium, static test rule, final-chain post-check.	End-to-end TCP and QUIC prototype; limitation demonstrated by alternate-path test.
1 — Correct enforcement	Request-scoped trust-store filtering, final postcondition, errors and diagnostics.	All path-building and unmatched-domain unit tests pass.
2 — QUIC/private trust	Explicit anchor source/local trust result, QUIC change and coalescing protections.	HTTP/3 matrix passes; arbitrary unknown root remains rejected.
3 — Dynamic policy	Immutable snapshots, cache/session invalidation and local signed bundle.	Update and expiry lifecycle tests pass.
4 — Central controller	Signed component distribution, rollback protection, CA rotation and onboarding.	Pilot fleet completes old/new rollover with no enforcement disablement.
5 — Productization	Private ungoogled fork, Linux/macOS packaging, signing, notarization and updater.	Stable channel meets security, conformance and operational SLOs.
6 — Hardening/upstreaming	Fuzzing, transparency log, formal review and candidate upstream design proposal.	Independent security assessment and documented long-term maintenance ownership.

15.2 Final recommendation

Use a private ungoogled-chromium patch stack for the maintained product and a direct Chromium checkout for rapid verifier development. Implement hostname-scoped trust-anchor filtering before path construction, retain a final-chain SPKI postcondition, and propagate an explicit trust-anchor source/disposition to QUIC and connection reuse. Do not disable QUIC globally and do not remove private trust globally. For hostnames with no matching rule, delegate to ordinary Chromium without semantic changes. For matching hostnames, make only approved anchors eligible.

Distribute the policy as an independently signed, versioned and expiring bundle rooted in a dedicated browser-policy authority. Automate CA rollover with old/new overlap and governed proof of current authority plus next-key possession. Automate domain onboarding through DNS control, CA key possession, Name Constraints validation, delegated namespace authority and audit-to-enforce promotion. The detailed issue backlog in Appendix C is organized so the prototype, correct verifier, QUIC support, centralized updater and product packaging can be delivered incrementally without weakening the target property.

Appendix A. Complete policy examples

A.1 Production bootstrap policy

```
{
 "DomainTrustPolicyEnabled": true,
 "DomainTrustPolicyUpdateUrl":
  "https://browser-policy.scone.cloud/components/domain-trust",
 "DomainTrustPolicyRootKeys": [
  "ed25519:BASE64_PRIMARY_OFFLINE_ROOT",
  "ed25519:BASE64_RECOVERY_OFFLINE_ROOT"
 ],
 "DomainTrustPolicyMaxStalenessSeconds": 604800,
 "DomainTrustPolicyExpiredMatchedDomainAction": "block",
 "DomainTrustPolicyAllowLocalTesting": false,
 "DomainTrustPolicyDiagnosticsEnabled": true
}
```

A.2 Production bundle with CA overlap

```
{
 "schema_version": 1,
 "policy_id": "scone-production",
 "generation": 184,
 "issued_at": "2026-07-26T05:00:00Z",
 "not_before": "2026-07-26T05:00:00Z",
 "expires_at": "2026-08-09T05:00:00Z",
 "minimum_browser_version": "136.0.0.0",
 "authorities": [
  {
   "id": "scone-ca-current",
   "spki_sha256": "sha256/BASE64_CURRENT",
   "der_base64": "BASE64_DER_CURRENT_CA",
   "trust_source": "bundle_private_anchor",
   "permitted_dns_subtrees": ["scone.cloud"]
  },
  {
   "id": "scone-ca-next",
   "spki_sha256": "sha256/BASE64_NEXT",
   "der_base64": "BASE64_DER_NEXT_CA",
   "trust_source": "bundle_private_anchor",
   "permitted_dns_subtrees": ["scone.cloud"]
  }
 ],
 "rules": [
  {
   "id": "scone-cloud-exclusive-ca",
   "dns_subtrees": ["scone.cloud"],
   "required_trust_anchor_ids": [
    "scone-ca-current", "scone-ca-next"
   ],
   "required_intermediate_spki_sha256": [],
   "mode": "enforce",
   "failure_mode": "fail_closed",
   "not_before": "2026-07-26T05:00:00Z",
   "not_after": "2027-01-31T23:59:59Z"
  }
 ],
 "revoked_authority_spkis": [],
 "next_policy_signing_keys": [],
 "signatures": [
  {
   "key_id": "production-online-policy-2026",
   "alg": "EdDSA",
   "value": "BASE64_SIGNATURE"
  }
 ]
}
```

A.3 Audit-mode onboarding bundle


```
{
 "schema_version": 1,
 "policy_id": "scone-production",
 "generation": 185,
 "authorities": [
  {
   "id": "tenant-a-ca",
   "spki_sha256": "sha256/BASE64_TENANT_A",
   "der_base64": "BASE64_DER_TENANT_A_CA",
   "trust_source": "bundle_private_anchor",
   "permitted_dns_subtrees": ["tenant-a.scone.cloud"]
  }
 ],
 "rules": [
  {
   "id": "tenant-a-audit",
   "dns_subtrees": ["tenant-a.scone.cloud"],
   "required_trust_anchor_ids": ["tenant-a-ca"],
   "mode": "audit",
   "failure_mode": "fail_closed",
   "not_before": "2026-08-01T00:00:00Z",
   "not_after": "2026-10-31T23:59:59Z"
  }
 ],
 "signatures": [{"key_id": "team-tenant-a", "alg": "EdDSA",
            "value": "BASE64_SIGNATURE"}]
}
```


A.4 Configuration lint rules

Table A-1 — Required linter checks before signing

Category	Checks
Encoding/signature	Canonical encoding; no duplicate keys; supported algorithms; valid signer delegation and threshold.
Time/version	Coherent timestamps; bounded lifetime; minimum browser version supported by target channel.
Authorities	SPKI matches DER; CA constraints; supported key; no duplicate/revoked SPKI; Name Constraints cover no more than declared scope.
Rules	Canonical DNS; no public suffix; no ambiguous equal-specificity overlap; every authority reference exists; enforce rules have non-empty anchor set.
Delegation	Signer scope contains every rule; validity and generation within delegation; no parent/sibling escape.
Operations	Rollover overlap within maximum; removal is not scheduled before fleet adoption gate; emergency revocation has incident reference.

Appendix B. Primary source-code integration map

Chromium moves code frequently. Before implementing against a new milestone, use source search and GN introspection to confirm the current path and owning target. The map below is intentionally responsibility-oriented.

Table B-1 — Discovery map for a pinned Chromium milestone

Responsibility	Search terms / likely path	Verification command
TCP certificate verification caller	SSLClientSocketImpl, CertVerifier::RequestParams, net/socket/ssl_client_socket_impl.cc	git grep "RequestParams" net/socket; gn refs out/Default <file>
QUIC proof verifier	ProofVerifierChromium, ERR_QUIC_CERT_ROOT_NOT_KNOWN, net/quic/crypto	git grep "ERR_QUIC_CERT_ROOT_NOT_KNOWN" net
Certificate result	CertVerifyResult, is_issued_by_known_root, public_key_hashes	git grep "struct CertVerifyResult" net
CertVerifier implementations	CertVerifier::CreateDefault, CachingCertVerifier, MultiThreadedCertVerifier	git grep "class .*CertVerifier" net services
Built-in path builder / trust store	SystemTrustStore, TrustStore, CertVerifyProcBuiltin, GetTrust	git grep "SystemTrustStore|GetTrust" net/cert third_party/boringssl
Enterprise certificate inputs	ProfileNetworkContextService, AdditionalCertificates, trust_anchors_with_additional_constraints	git grep "trust_anchors_with_additional_constraints" chrome services
Verifier service Mojo	CertVerifierServiceUpdater, cert_verifier.mojom	git grep "CertVerifierServiceUpdater" .
Policy templates	policy_definitions, generated policy schema, supported_on	find components/policy -iname "*Certificate*" -o -iname "*policy*"
Component updater	ComponentInstallerPolicy, ComponentUpdateService	git grep "ComponentInstallerPolicy" components chrome
HTTP/2/QUIC pools	HttpNetworkSession, SpdySessionPool, QuicSessionPool	git grep "SessionPool" net/http net/quic
NetLog	NetLogEventType, CERT_VERIFIER, QUIC proof events	git grep "CERT_VERIFIER" net/log net/cert

B.1 Expected patch boundaries

- Policy schema and browser preference plumbing should be separate from net-stack semantics.
- Policy bundle cryptography/parser should be independently fuzzable and should not depend on UI code.
- Trust-store filtering should have no dependency on Chrome profile objects; it consumes a compiled immutable net-layer snapshot.
- QUIC receives an explicit result field and does not reimplement hostname matching.
- Session/cache invalidation consumes a policy-diff object rather than reparsing the signed bundle.
- Packaging patches should contain branding, application IDs, updater endpoints and signing configuration—not verifier logic.

Appendix C. Issue-ready implementation backlog

The following items are written so they can be copied into an issue tracker. Estimates are intentionally omitted because they depend strongly on the selected Chromium milestone and team familiarity. Suggested labels provide grouping, not priority.

C.1 Fork, build and release foundation

FORK-01 — Create private common ungoogled-chromium fork

Objective: Establish the authoritative private repository that carries the domain-trust patch series.

Implementation tasks:

- Fork the common ungoogled-chromium repository.
- Add read-only upstream and Chromium remotes.
- Protect vendor, release and main branches.
- Document signed-tag and review policy.

Acceptance criteria:

- A clean vendor branch matches the selected upstream tag.
- Pushes to upstream remotes are disabled.
- Two-person review is required for patches/secure-domain-trust and signing configuration.

Dependencies: None | Suggested labels: fork, supply-chain

FORK-02 — Create private portable-Linux packaging fork

Objective: Produce reproducible Linux packages from the private common fork.

Implementation tasks:

- Fork portablelinux with submodules.
- Point the common submodule to browser-common.
- Pin the Docker builder image by digest.
- Add artifact, SBOM and provenance publication.

Acceptance criteria:

- scripts/docker-build.sh -c builds from a clean worker.
- scripts/package.sh creates AppImage and tar.xz.
- Artifact manifest records every source and toolchain revision.

Dependencies: FORK-01 | Suggested labels: linux, build

FORK-03 — Create private macOS packaging fork

Objective: Produce arm64 and required x86-64 macOS artifacts from the private common fork.

Implementation tasks:

- Fork macOS packaging with submodules.
- Configure private application identity and updater endpoint.
- Add controlled Developer ID signing and notarization jobs.
- Pin Xcode/SDK requirements by Chromium milestone.

Acceptance criteria:

- Native build produces an installable app/DMG.
- codesign, notarization and Gatekeeper assessment pass.
- Common submodule and patch revision are recorded.

Dependencies: FORK-01 | Suggested labels: macos, build, signing

FORK-04 — Establish vendor import and rebase automation

Objective: Import each new ungoogled/Chromium milestone without losing provenance.

Implementation tasks:

- Verify upstream tag and source digests.
- Create vendor branch and signed import tag.
- Open automated rebase branches in common and packaging forks.

- Generate patch-application and conflict reports.

Acceptance criteria:

- No source is imported without a verified origin/digest.
- Pipeline stops on patch fuzz or unresolved conflict.
- Release manifest links upstream and downstream revisions.

Dependencies: FORK-01,FORK-02,FORK-03 | Suggested labels: automation, rebase

FORK-05 — Create zero-fuzz private patch group

Objective: Isolate every verifier feature change under patches/secure-domain-trust.

Implementation tasks:

- Create ordered patch naming convention.
- Append group to patches/series after inherited patches.
- Add validate_patches and check_patch_files gates.
- Document patch ownership.

Acceptance criteria:

- Every patch applies with zero fuzz to the pinned source.
- Series order is deterministic and reviewed.
- CI detects untracked or orphan patch files.

Dependencies: FORK-01 | Suggested labels: patches, quilt

FORK-06 — Provision deterministic Linux builders

Objective: Build Linux artifacts on ephemeral, pinned workers.

Implementation tasks:

- Create builder image with pinned base digest and dependencies.
- Populate an integrity-checked dependency mirror.
- Restrict network egress during compile.
- Publish builder SBOM.

Acceptance criteria:

- Two clean builds use identical source/tool inputs.
- Builder image digest is in provenance.
- No undeclared network dependency is required.

Dependencies: FORK-02 | Suggested labels: linux, reproducibility

FORK-07 — Provision deterministic macOS builders

Objective: Build and notarize macOS artifacts on controlled workers.

Implementation tasks:

- Create clean macOS VM images for supported versions.
- Install pinned Xcode/SDK and dependencies.
- Integrate hardware-backed signing access only in release stage.
- Archive notarization records.

Acceptance criteria:

- Unsigned build is reproducible within documented limits.
- Release job signs only reviewed artifacts.
- Notarization and stapling are automatically verified.

Dependencies: FORK-03 | Suggested labels: macos, reproducibility

FORK-08 — Add source, license and SBOM generation

Objective: Produce complete third-party notices and machine-readable component inventories.

Implementation tasks:

- Run Chromium license generation.
- Collect downstream and private patch licenses.
- Generate CycloneDX or SPDX SBOM for package and builder.
- Fail on missing license metadata.

Acceptance criteria:

- Every shipped binary has a corresponding license bundle and SBOM.
- SBOM identifies Chromium milestone, downstream revision and custom patch revision.

Dependencies: FORK-02,FORK-03 | Suggested labels: compliance, sbom

FORK-09 — Create multi-channel update infrastructure

Objective: Distribute dev, pilot, stable and recovery browser builds securely.

Implementation tasks:

- Define application IDs and channel endpoints.
- Sign update metadata.
- Implement staged rollout and rollback controls.
- Test code-signing key rotation.

Acceptance criteria:

- Clients cannot cross channels without managed configuration.
- Rollback to an older vulnerable build is blocked except by authorized recovery.
- Pilot promotion is auditable.

Dependencies: FORK-02,FORK-03 | Suggested labels: updates, release

FORK-10 — Add build provenance and release manifest

Objective: Bind every package to source, patches, tools and tests.

Implementation tasks:

- Generate signed provenance from the release job.
- Record patch hashes, toolchain versions and policy schema.
- Attach conformance and vulnerability reports.
- Publish immutable release manifest.

Acceptance criteria:

- A verifier can trace an installed package to exact source inputs.
- Manifest signature is validated before promotion.

Dependencies: FORK-06,FORK-07,FORK-08 | Suggested labels: provenance, release

FORK-11 — Create upstream-Chromium prototype workspace

Objective: Maintain a minimal direct Chromium environment for rapid net-stack experiments.

Implementation tasks:

- Automate fetch/gclient/gn setup.
- Add a static test rule build flag.
- Build focused net and QUIC tests.
- Document mapping from prototype commits to Quilt patches.

Acceptance criteria:

- A developer can compile and run a focused verifier test without packaging repos.
- Prototype changes are never shipped directly.

Dependencies: None | Suggested labels: prototype, chromium

FORK-12 — Define emergency Chromium security rebase process

Objective: Integrate critical upstream security releases within the organization's SLO.

Implementation tasks:

- Define intake trigger and on-call ownership.
- Create shortened mandatory critical test set.
- Automate package signing/pilot deployment.
- Document protected-service contingency if rebase fails.

Acceptance criteria:

- A tabletop exercise completes end to end.
- No emergency path permits skipping anchor/QUIC conformance tests.

Dependencies: FORK-04,FORK-09,FORK-10 | Suggested labels: security-release, operations

C.2 Policy model and parser

POL-01 — Define DomainTrustPolicy schema and semantics

Objective: Specify the versioned atomic bundle and normative matching/enforcement behavior.

Implementation tasks:

- Define JSON/CBOR schema.
- Document every field and default.
- Define audit/enforce and failure behavior.
- Define compatibility/versioning rules.

Acceptance criteria:

- Schema represents authorities, rules, revocation, delegation, generation and signatures.
- No rule can become active with an unresolved authority reference.

Dependencies: FORK-11 | Suggested labels: policy, design

POL-02 — Implement hostname canonicalization

Objective: Convert reference hostnames and policy subtrees to a single safe canonical form.

Implementation tasks:

- Use Chromium canonicalization/IDNA primitives.
- Handle trailing dot and case.
- Reject invalid DNS and IP literals for DNS rules.
- Add fuzz target.

Acceptance criteria:

- Apex/subdomain/suffix-confusion test corpus passes.
- Canonicalization is identical in compiler and verifier lookup.

Dependencies: POL-01 | Suggested labels: policy, dns, fuzzing

POL-03 — Implement most-specific rule compiler

Objective: Compile canonical subtrees into deterministic longest-match lookup.

Implementation tasks:

- Choose immutable trie or sorted suffix structure.
- Reject equal-specificity ambiguity.
- Return rule ID and digest.
- Benchmark large rule sets.

Acceptance criteria:

- Lookup is complete-label and deterministic.
- No-rule lookup has bounded low overhead.
- 10,000-rule benchmark meets agreed budget.

Dependencies: POL-02 | Suggested labels: policy, performance

POL-04 — Implement authority validation

Objective: Validate SPKI identifiers and optional DER private anchors before activation.

Implementation tasks:

- Parse DER and compute SPKI.
- Check CA/basic constraints, key use and algorithms.
- Validate embedded/external Name Constraints.
- Reject duplicate and revoked SPKIs.

Acceptance criteria:

- DER/SPKI mismatch is rejected.
- A bundle anchor cannot escape declared DNS scope.
- Explicit distrust remains effective.

Dependencies: POL-01 | Suggested labels: policy, pki

POL-05 — Implement policy lint and overlap checks

Objective: Reject unsafe scopes and operationally invalid rollovers before signing or activation.

Implementation tasks:

- Integrate Public Suffix List checks.
- Detect parent/sibling overlap and delegation escape.
- Validate time windows and overlap limits.
- Produce actionable diagnostics.

Acceptance criteria:

- A rule for cloud is rejected by default.
- A delegated key for dev.scone.cloud cannot sign prod.scone.cloud.
- Invalid candidate never replaces last-known-good.

Dependencies: POL-01,POL-03,POL-04 | Suggested labels: policy, governance

POL-06 — Add enterprise bootstrap policies

Objective: Define mandatory browser policies for enabling and bootstrapping signed updates.

Implementation tasks:

- Add policy template definitions.
- Add preferences and handlers.
- Document supported platforms and dynamic refresh.
- Expose status in chrome://policy.

Acceptance criteria:

- Policies appear with correct source/scope/status.
- User policy cannot override mandatory machine roots.
- Disabling local testing is effective in production builds.

Dependencies: POL-01,FORK-05 | Suggested labels: enterprise-policy, chrome

POL-07 — Implement immutable policy snapshot service

Objective: Publish validated policy state atomically to verifier clients.

Implementation tasks:

- Compile candidate off-thread.
- Use immutable reference-counted snapshot.
- Atomic activation with generation.
- Retain last-known-good and diagnostics.

Acceptance criteria:

- Concurrent requests see either old or new complete snapshot.
- Malformed candidate leaves current state untouched.
- Snapshot contains no mutable parser objects.

Dependencies: POL-03,POL-04,POL-05 | Suggested labels: policy, concurrency

POL-08 — Add policy diagnostics and NetLog events

Objective: Make effective behavior supportable without exposing sensitive policy content.

Implementation tasks:

- Define structured update/match/error events.
- Add enterprise diagnostics page.
- Expose generation, signer, expiry and rule ID.
- Redact DER and URLs.

Acceptance criteria:

- Support can identify why a connection was blocked.
- Web content and ordinary extensions cannot read protected policy details.

Dependencies: POL-06,POL-07 | Suggested labels: diagnostics, netlog

C.3 Certificate verifier implementation

VER-01 — Add domain trust disposition to CertVerifyResult

Objective: Carry the policy and trust-anchor decision to TCP, QUIC, sessions and diagnostics.

Implementation tasks:

- Define TrustAnchorSource and DomainTrustDisposition.
- Add copy/reset/equality behavior.
- Populate final anchor SPKI and local/private status.
- Update Mojo serialization if result crosses process boundaries.

Acceptance criteria:

- Round-trip tests preserve all fields.
- Default result is no-rule/unknown and cannot imply local trust.
- No sensitive certificate DER is added to generic telemetry.

Dependencies: POL-07 | Suggested labels: cert-verifier, api

VER-02 — Add dedicated certificate errors and status flags

Objective: Distinguish domain-anchor enforcement failures from ordinary path errors.

Implementation tasks:

- Add required-anchor and required-intermediate net errors.
- Add certificate status flags and mappings.
- Update SSL interstitial and error strings.
- Add NetLog serialization.

Acceptance criteria:

- Matching-domain unapproved chains return the dedicated error.
- Certificate-policy failures are fatal and have no user bypass in enforce mode.

Dependencies: VER-01 | Suggested labels: cert-verifier, errors, ux

VER-03 — Implement PolicyFilteringTrustStore

Objective: Filter positive trust anchors by allowed SPKI before path construction.

Implementation tasks:

- Wrap the aggregate SystemTrustStore/BoringSSL TrustStore.
- Delegate issuer enumeration.
- Return unspecified for unapproved positive anchors.
- Preserve explicit distrust and implement all current TrustStore virtual methods.

Acceptance criteria:

- Unapproved roots cannot terminate a path for a matching rule.
- Approved roots remain usable.
- Unmatched requests never instantiate the filter.

Dependencies: POL-07,VER-01 | Suggested labels: cert-verifier, trust-store

VER-04 — Integrate request-scoped filter into built-in verifier

Objective: Select the filtered trust view using the canonical reference hostname.

Implementation tasks:

- Capture immutable snapshot at request start.
- Look up most-specific rule.
- Compose browser-managed private anchors with baseline trust.
- Pass filtered store to path builder.

Acceptance criteria:

- All path building for a matched request uses one snapshot.
- No caller-provided Host header can change rule selection.
- Baseline no-rule behavior is bit-for-bit equivalent where practical.

Dependencies: POL-03,POL-07,VER-03 | Suggested labels: cert-verifier, integration

VER-05 — Add final-chain security postcondition

Objective: Verify that the selected path actually contains the authorized anchor and optional intermediate.

Implementation tasks:

- Compute final anchor SPKI from verified path.
- Compare against resolved rule.
- Evaluate optional intermediate list.
- Attach disposition or return dedicated error.

Acceptance criteria:

- A deliberately faulty trust filter is caught by the postcondition test.
- Old/new anchor OR semantics work.
- Intermediate requirement is independent from anchor requirement.

Dependencies: VER-04,VER-02 | Suggested labels: cert-verifier, defense-in-depth

VER-06 — Classify trust-anchor source

Objective: Reliably identify Chrome public, platform/user, enterprise and browser-managed anchors.

Implementation tasks:

- Trace trust-source information through aggregate trust store.
- Add explicit browser-managed source.
- Handle test verifier sources.
- Document unknown/fallback behavior.

Acceptance criteria:

- A local macOS/Linux root is classified as local/private.
- A Chrome Root Store root is not mislabeled local.
- Unknown source never enables QUIC private-root exception.

Dependencies: VER-03,VER-04 | Suggested labels: cert-verifier, trust-source

VER-07 — Add authenticated browser-managed private-anchor store

Objective: Permit signed bundles to supply private CA certificates without modifying global platform trust.

Implementation tasks:

- Parse validated authority DER into a dedicated store.
- Intersect Name Constraints and rule scope.
- Compose with baseline store.
- Remove anchors atomically on policy update.

Acceptance criteria:

- Only signed, validated bundle authorities are trusted.
- An SPKI identifier without authenticated DER does not trust a self-signed server cert.
- Removal affects new path builds immediately.

Dependencies: POL-04,POL-07,VER-03 | Suggested labels: private-pki, trust-store

VER-08 — Intersect existing and domain constraints

Objective: Ensure the new rule never widens an anchor's embedded or externally configured constraints.

Implementation tasks:

- Preserve embedded critical Name Constraints.
- Preserve CACertificatesWithConstraints restrictions.
- Apply domain rule as an additional intersection.
- Add conflicting-constraint tests.

Acceptance criteria:

- A CA allowed only for dev.scone.cloud cannot be used for prod.scone.cloud even if a broad rule lists it.
- Excluded/explicitly distrusted names remain blocked.

Dependencies: VER-04,VER-07 | Suggested labels: name-constraints, verifier

VER-09 — Support audit mode

Objective: Measure future enforcement without altering baseline connection acceptance.

Implementation tasks:

- Run baseline verification.
- Evaluate final chain against rule.
- Record compliant/noncompliant outcome.
- Prevent audit data from enabling local trust.

Acceptance criteria:

- Audit mode never changes the underlying success/error.
- Telemetry distinguishes no path, wrong anchor and wrong intermediate.
- Switching to enforce is classified as tightening.

Dependencies: VER-01,VER-05,POL-08 | Suggested labels: audit, telemetry

VER-10 — Make verifier cache policy-aware

Objective: Prevent cached certificate results from bypassing a new or changed rule.

Implementation tasks:

- Clear cache on effective generation change for first release.
- Record generation in cached result.
- Add tests for no-rule to enforce transition.
- Design optional rule-digest optimization.

Acceptance criteria:

- A result cached before rule activation is never reused after activation.
- Malformed candidate does not clear current state unnecessarily.
- Unmatched behavior remains performant.

Dependencies: VER-04,POL-07 | Suggested labels: cache, verifier

VER-11 — Add verifier performance benchmarks

Objective: Bound overhead for unmatched and matched requests.

Implementation tasks:

- Benchmark no-rule lookup.
- Benchmark filtered GetTrust and path building.
- Test 10,000 rules and large trust stores.
- Track allocations and latency in CI.

Acceptance criteria:

- Unmatched overhead meets agreed microbenchmark budget.
- Matched path remains within operational handshake target.
- No linear scan of all rules per request.

Dependencies: POL-03,VER-04 | Suggested labels: performance, benchmark

VER-12 — Add verifier security review checklist

Objective: Require explicit review of security invariants on every rebase.

Implementation tasks:

- Document trust-store interfaces and assumptions.
- Map each invariant I-1–I-10 to code/tests.
- Add reviewer checklist to PR template.
- Require network-security owner approval.

Acceptance criteria:

- Every release records checklist completion.
- Interface changes in BoringSSL/Chromium trigger a blocking review task.

Dependencies: VER-03,VER-04,VER-05 | Suggested labels: security-review, process

C.4 QUIC, sessions and transport integration

NET-01 — Extend QUIC proof verifier for legitimate local trust

Objective: Permit successful local/private trust over QUIC without accepting arbitrary unknown roots.

Implementation tasks:

- Consume DomainTrustDisposition locally_trusted_anchor.

- Retain known-root and host-exception behavior.
- Return required-anchor errors unchanged.
- Add focused proof-verifier tests.

Acceptance criteria:

- Unmatched R-local succeeds over QUIC.
- Unknown self-signed root fails.
- Matched unapproved R-local fails before/at domain policy, not via a bypass.

Dependencies: VER-01,VER-06 | Suggested labels: quic, http3, private-pki

NET-02 — Propagate policy identity to SSL and QUIC sessions

Objective: Bind established connections to the rule and generation under which they were authenticated.

Implementation tasks:

- Add rule ID/digest, generation and anchor SPKI to session metadata.
- Expose to pool reuse checks.
- Avoid storing full policy bundle.
- Add serialization/lifetime tests.

Acceptance criteria:

- Every matched live session can be classified during an update.
- No-rule sessions carry an explicit baseline equivalence marker.

Dependencies: VER-01,VER-05 | Suggested labels: sessions, metadata

NET-03 — Make HTTP/2 connection coalescing policy-aware

Objective: Prevent a connection authenticated under one rule from serving an incompatible origin.

Implementation tasks:

- Compute policy equivalence for candidate origin.
- Re-evaluate final chain against candidate rule.
- Deny coalescing on generation/rule mismatch.
- Add NetLog reason.

Acceptance criteria:

- SAN overlap cannot bypass a stricter child-domain rule.
- Equivalent same-rule origins retain baseline coalescing where otherwise valid.

Dependencies: NET-02,POL-03 | Suggested labels: http2, coalescing

NET-04 — Make QUIC connection coalescing policy-aware

Objective: Apply the same origin-equivalence rules to QUIC sessions.

Implementation tasks:

- Integrate rule equivalence into QUIC session pool lookup.
- Re-check certificate disposition for candidate origin.
- Test migration and pooled reuse.
- Emit diagnostics.

Acceptance criteria:

- Different rule IDs/generations never share a QUIC session.
- Same-rule sharing still requires all normal QUIC checks.

Dependencies: NET-01,NET-02,POL-03 | Suggested labels: quic, coalescing

NET-05 — Invalidate sessions on tightening policy changes

Objective: Stop using connections established under a no-longer-valid anchor or absent rule.

Implementation tasks:

- Compile policy diff with affected subtrees/SPKIs.
- Mark HTTP/1, HTTP/2 and QUIC sessions non-reusable.
- Close emergency-revoked sessions immediately.
- Test concurrent traffic.

Acceptance criteria:

- Removed-anchor sessions are not reused after activation.

- Loosening changes do not cause unnecessary fleet-wide disconnect.
- Emergency revocation closes all known affected sessions.

Dependencies: NET-02,POL-07 | Suggested labels: sessions, invalidation

NET-06 — Bind TLS resumption to policy equivalence

Objective: Prevent session tickets from bypassing a changed rule.

Implementation tasks:

- Include policy equivalence/generation in cache key or metadata.
- Invalidate on tightening.
- Revalidate candidate origin before resumption.
- Cover browser restart persistence.

Acceptance criteria:

- Ticket from no-rule state cannot resume after subtree becomes enforced.
- Old-CA ticket cannot resume after CA removal.

Dependencies: NET-02,NET-05 | Suggested labels: tls, resumption

NET-07 — Define 0-RTT policy for protected origins

Objective: Ensure early data is not sent under stale certificate-policy state.

Implementation tasks:

- Disable 0-RTT for matched origins in first release.
- Preserve baseline for unmatched origins.
- Add future design for generation-bound safe 0-RTT.
- Test policy update between ticket issue and use.

Acceptance criteria:

- No early data is transmitted for a matched origin before compliance is established.
- Unmatched QUIC behavior remains baseline.

Dependencies: NET-06,NET-01 | Suggested labels: quic, 0rtt

NET-08 — Prevent certificate-policy fallback bypass

Objective: Ensure an HTTP/3 certificate-policy failure cannot succeed by retrying over TCP.

Implementation tasks:

- Classify certificate-policy errors as transport-independent.
- Audit fallback decision paths.
- Add H3-to-H2/H1 tests.
- Preserve fallback for non-certificate transport errors.

Acceptance criteria:

- Required-anchor failure is final across transports.
- Network-only QUIC failure may still fall back normally.

Dependencies: NET-01,VER-02 | Suggested labels: quic, fallback

C.5 Test and conformance work packages

TST-01 — Create deterministic multi-root test PKI

Objective: Generate reproducible approved, public, local, interception, revoked and cross-signed test chains.

Implementation tasks:

- Write declarative PKI scenarios.
- Generate roots/intermediates/leaves and fingerprints.
- Create OCSP/CRL fixtures.
- Mark all keys test-only.

Acceptance criteria:

- One command regenerates identical semantic scenarios.
- Every certificate purpose and expected path is documented.
- No production key can enter the repository.

Dependencies: None | Suggested labels: testing, pki

TST-02 — Add policy parser and matcher unit tests

Objective: Cover canonicalization, schema, overlap and rule selection.

Implementation tasks:

- Implement table-driven DNS corpus.
- Test malformed and conflicting bundles.
- Test delegation scope.
- Add regression fixtures for every discovered bug.

Acceptance criteria:

- All invariant I-2 cases are covered.
- Equal-specificity conflict is rejected.
- Public-suffix rule is rejected by default.

Dependencies: POL-02,POL-03,POL-05 | Suggested labels: testing, policy

TST-03 — Add trust-store filter unit tests

Objective: Prove anchor filtering and distrust precedence independently of browser UI.

Implementation tasks:

- Construct fake aggregate trust stores.
- Exercise GetTrust and issuer delegation.
- Test bundle private anchors.
- Test current BoringSSL TrustStore methods.

Acceptance criteria:

- Unapproved positive trust becomes unspecified.
- Explicit distrust always wins.
- Issuer candidates remain discoverable.

Dependencies: VER-03,VER-07,TST-01 | Suggested labels: testing, trust-store

TST-04 — Add alternate-path and cross-sign verifier tests

Objective: Demonstrate why pre-path filtering is required and verify approved alternate path selection.

Implementation tasks:

- Create leaf/intermediate with paths to approved and unapproved roots.
- Run post-check-only control test.
- Run filtered-path test.
- Cover AIA and cached intermediates.

Acceptance criteria:

- Control test exhibits false rejection or wrong path.
- Production filter builds the approved path when available.
- No approved path yields dedicated error.

Dependencies: VER-04,VER-05,TST-01 | Suggested labels: testing, path-building

TST-05 — Add QUIC private-root matrix

Objective: Verify public, local-private, approved and unknown root behavior over HTTP/3.

Implementation tasks:

- Create QUIC test server scenarios.
- Exercise ProofVerifierChromium directly.
- Exercise browser HTTP/3.
- Test fallback behavior.

Acceptance criteria:

- All rows in Table 13 pass.
- Arbitrary unknown root remains rejected.
- Matched unapproved local root never succeeds.

Dependencies: NET-01,NET-08,TST-01 | Suggested labels: testing, quic

TST-06 — Add browser-surface integration suite

Objective: Apply the same rule to navigation, fetch, WebSocket, service workers and redirects.

Implementation tasks:

- Build test pages and servers.
- Add WebSocket reconnect after update.
- Add service-worker background fetch.
- Add preconnect and redirect cases.

Acceptance criteria:

- Every surface returns the same policy decision.
- No stale worker or reconnect bypasses tightening.

Dependencies: VER-04,NET-05,TST-01 | Suggested labels: testing, browser

TST-07 — Add pooling, coalescing and resumption suite

Objective: Prove that connection reuse cannot cross incompatible policy state.

Implementation tasks:

- Test HTTP/2 and QUIC SAN coalescing.
- Test no-rule to rule activation.
- Test old CA removal.
- Test browser restart/session persistence.

Acceptance criteria:

- Incompatible origin/session reuse is denied.
- Equivalent origins retain permitted reuse.
- Resumption tickets are invalidated on tightening.

Dependencies: NET-03,NET-04,NET-06 | Suggested labels: testing, sessions

TST-08 — Add signed-update lifecycle suite

Objective: Verify authenticity, atomicity, expiry, rollback and key rotation.

Implementation tasks:

- Build local component/update server.
- Inject bad signatures and partial writes.
- Replay old generations.
- Exercise dual-sign policy-key rotation.

Acceptance criteria:

- Invalid candidate never replaces last-known-good.
- Expired matching rules fail closed while unmatched browsing remains baseline.
- Rollback attempts are logged and rejected.

Dependencies: CFG-01,CFG-02,CFG-03 | Suggested labels: testing, updater

TST-09 — Add Linux and macOS platform-trust tests

Objective: Validate locally added roots on both target platforms and transports.

Implementation tasks:

- Provision disposable Linux trust/NSS environment as applicable.
- Provision temporary macOS keychain/profile.
- Add R-local and R-intercept.
- Run matched/unmatched TCP and QUIC matrix.

Acceptance criteria:

- Unmatched local private trust works on both platforms.
- Protected namespace rejects unapproved platform roots.
- Cleanup leaves no trust residue.

Dependencies: NET-01,VER-06,TST-01 | Suggested labels: testing, linux, macos

TST-10 — Add fuzzing, differential and mutation testing

Objective: Find parser/matcher bugs and prove tests detect fail-open mutations.

Implementation tasks:

- Create libFuzzer targets for bundle and DNS input.
- Differentially compare no-rule results with baseline Chromium.
- Inject branch mutations in allow/deny logic.
- Run corpus continuously.

Acceptance criteria:

- No-rule differential corpus shows no unexplained acceptance changes.
- Mutations that skip filter/postcondition are detected by CI.
- Crashes produce minimized regression cases.

Dependencies: POL-02,POL-04,VER-03,VER-05 | Suggested labels: fuzzing, differential-testing

C.6 Signed configuration, controller and operations

CFG-01 — Define canonical signed bundle format

Objective: Specify deterministic bytes, signatures and environment binding.

Implementation tasks:

- Select canonical CBOR/COSE or canonical JSON/JWS profile.
- Define duplicate-key and normalization rules.
- Define signature threshold and key IDs.
- Publish test vectors.

Acceptance criteria:

- Independent implementations produce identical signed bytes.
- Staging bundle cannot validate in production.
- Malformed canonicalization is rejected before semantics.

Dependencies: POL-01 | Suggested labels: controller, cryptography

CFG-02 — Implement browser-side signature and delegation verifier

Objective: Authenticate bundles independently of HTTPS transport.

Implementation tasks:

- Embed bootstrap root keys from mandatory policy.
- Verify delegation chain, scope and time.
- Support threshold signatures.
- Return structured failure diagnostics.

Acceptance criteria:

- Bundle signed only by TLS server key is rejected.
- Out-of-scope delegate is rejected.
- Valid root/online/delegate chain is accepted.

Dependencies: CFG-01,POL-06 | Suggested labels: browser, signatures

CFG-03 — Implement rollback-protected last-known-good storage

Objective: Persist the active bundle and monotonic state safely across crashes/restarts.

Implementation tasks:

- Store highest generation/key epoch.
- Use temporary write, fsync and atomic rename.
- Retain previous snapshots for diagnostics.
- Protect permissions and integrity.

Acceptance criteria:

- Power-loss injection never activates partial policy.
- Older validly signed generation is rejected.
- Browser restarts with the last complete snapshot.

Dependencies: CFG-02,POL-07 | Suggested labels: storage, rollback

CFG-04 — Integrate private component updater

Objective: Fetch and stage signed domain-trust bundles through a controlled update channel.

Implementation tasks:

- Register component and update endpoint.
- Implement cadence, backoff and staged rollout.
- Verify content signature after transport.
- Expose component version/health.

Acceptance criteria:

- Transport compromise cannot activate unsigned content.
- Update outage preserves last-known-good.
- Component can be rolled through dev/pilot/stable.

Dependencies: CFG-02,CFG-03,FORK-09 | Suggested labels: component-updater, browser

CFG-05 — Build domain-trust policy compiler service

Objective: Convert approved authorities, rules and delegations into a signed bundle.

Implementation tasks:

- Implement database/model and linter.
- Resolve authority IDs and compile canonical payload.
- Sign with HSM-backed online key.
- Record immutable audit event.

Acceptance criteria:

- Compiler cannot emit an internally inconsistent bundle.
- Every generation is reproducible from approved inputs.
- Signing key use is audited.

Dependencies: CFG-01,POL-05 | Suggested labels: controller, compiler

CFG-06 — Implement CA key-possession and current-authority rollover proofs

Objective: Authenticate routine CA rotation requests without making the TLS CA the sole policy authority.

Implementation tasks:

- Issue nonce challenge.
- Verify current-authority signature or registered rollover key.
- Verify next-CA private-key possession.
- Bind proofs to namespace and proposed times.

Acceptance criteria:

- Replayed proof is rejected.
- Proof for one namespace cannot rotate another.
- Final bundle still requires organizational policy signature.

Dependencies: CFG-05,POL-04 | Suggested labels: controller, ca-rotation

CFG-07 — Automate old/new CA overlap workflow

Objective: Safely publish, observe and retire CA anchors.

Implementation tasks:

- Generate overlap candidate.
- Gate promotion on pilot/fleet adoption.
- Integrate service cutover status.

- Publish tightening removal and session-invalidation notice.

Acceptance criteria:

- No workflow step disables enforcement.
- Old and next CA are both accepted only during approved window.
- Old removal cannot occur before adoption gate without emergency approval.

Dependencies: CFG-06,CFG-04,NET-05 | Suggested labels: controller, ca-rotation

CFG-08 — Implement emergency SPKI revocation

Objective: Remove a compromised CA rapidly and close affected sessions.

Implementation tasks:

- Support offline/threshold emergency signing.
- Publish generation jump and revoked SPKI.
- Trigger high-priority client update.
- Integrate incident ticket and audit.

Acceptance criteria:

- Revoked SPKI is distrusted even if still listed by a rule.
- Affected live sessions close.
- Emergency action is visible in fleet telemetry.

Dependencies: CFG-02,CFG-05,NET-05 | Suggested labels: incident-response, revocation

CFG-09 — Implement DNS control and domain onboarding workflow

Objective: Add new protected subtrees and their CAs through governed automation.

Implementation tasks:

- Generate DNS TXT challenge.
- Verify organizational requester and DNS proof.
- Verify CA key possession and constraints.
- Create audit-mode candidate and tests.

Acceptance criteria:

- Public suffix and unauthorized overlap are rejected.
- New rule is audit before enforce unless emergency exception is approved.
- Onboarding record includes contacts and expiry.

Dependencies: CFG-05,POL-05,TST-01 | Suggested labels: controller, onboarding

CFG-10 — Implement constrained namespace delegation

Objective: Let teams manage their own subtrees without parent/sibling authority.

Implementation tasks:

- Define delegation object and signing flow.
- Enforce DNS/time/environment scope.
- Enforce maximum overlap and rule validity.
- Support revocation and renewal.

Acceptance criteria:

- dev.scone.cloud delegate cannot change scone.cloud or prod.scone.cloud.
- Expired/revoked delegate cannot sign an accepted bundle.
- Delegated generations are monotonic.

Dependencies: CFG-01,CFG-05,POL-05 | Suggested labels: controller, delegation

CFG-11 — Add policy transparency and audit log

Objective: Make every effective policy generation and authorization path reviewable.

Implementation tasks:

- Store bundle hash, signer chain and approvals.
- Optionally publish Merkle-log entries.
- Provide diff and rollback investigation UI.
- Restrict access to private namespace data.

Acceptance criteria:

- Every client-visible generation has an immutable audit record.
- Unauthorized equivocation between channels is detectable.
- Sensitive data is access controlled.

Dependencies: CFG-05,CFG-04 | Suggested labels: audit, transparency

CFG-12 — Create fleet policy-health dashboard and alerts

Objective: Track adoption, expiry, failures and noncompliant certificate use.

Implementation tasks:

- Ingest privacy-bounded browser metrics.
- Show generation and browser-version distribution.
- Alert on signature/rollback/expiry errors.
- Gate CA rotation on adoption thresholds.

Acceptance criteria:

- Operators can identify clients missing overlap before server cutover.
- Dashboard does not ingest page content or credentials.
- Alerts link to runbooks.

Dependencies: POL-08,CFG-04,CFG-07 | Suggested labels: operations, telemetry

Appendix D. Conformance test matrix

Table D-1 — Minimum cross-layer conformance suite

ID	Area	Scenario	Expected result
C-01	DNS match	scone.cloud apex with scone.cloud rule	Rule matches.
C-02	DNS match	api.scone.cloud	Rule matches.
C-03	DNS match	east.api.scone.cloud	Rule matches.
C-04	DNS match	evil-scone.cloud	No match; baseline behavior.
C-05	DNS match	SCONE.CLOUD. with case/trailing dot	Canonical rule matches.
C-06	DNS match	Valid IDN represented as U-label/A-label	Same canonical rule result.
C-07	Rule selection	Parent and child rules both match	Most-specific child rule wins.
C-08	Rule selection	Equal-specificity conflicting rules	Candidate policy rejected.
C-09	Policy lint	Rule for public suffix cloud	Rejected by default.
C-10	Baseline	Unmatched public site using public root	TCP and QUIC baseline success.
C-11	Baseline	Unmatched intranet using installed private root	TCP and QUIC success.
C-12	Baseline	Unmatched self-signed unknown root	TCP and QUIC reject.
C-13	Protected path	Approved root, valid leaf	TCP and QUIC success.
C-14	Protected path	Unapproved public root, valid leaf	Required-anchor failure.
C-15	Protected path	Unapproved local/interception root	Required-anchor failure.
C-16	Protected path	Approved root explicitly distrusted	Reject; distrust wins.
C-17	Protected path	Approved root outside embedded Name Constraints	Reject.
C-18	Protected path	Wrong hostname/SAN under approved root	Ordinary hostname error.
C-19	Protected path	Expired leaf under approved root	Ordinary expiry error.
C-20	Protected path	Revoked leaf/chain under approved root	Revocation result per configured policy.
C-21	Alternate path	Approved and unapproved valid paths	Approved path is constructed and accepted.
C-22	Alternate path	Only unapproved path exists	Required-anchor failure.
C-23	Alternate path	Approved path requires AIA intermediate	Approved path succeeds when AIA is allowed/available.
C-24	Intermediate	Required intermediate present	Success.
C-25	Intermediate	Approved root but required intermediate absent	Required-intermediate failure.
C-26	Audit	Unapproved path in audit mode	Baseline acceptance plus noncompliant audit event.
C-27	QUIC	Approved private root for matched domain	HTTP/3 succeeds.
C-28	QUIC	Legitimate local root for unmatched domain	HTTP/3 succeeds.
C-29	QUIC	Unknown root not locally trusted	ERR_QUIC_CERT_ROOT_NOT_KNOWN or ordinary trust error.
C-30	Fallback	HTTP/3 required-anchor failure	No successful TCP fallback.
C-31	Coalescing	Same cert, origins governed by different rules	No HTTP/2 or QUIC coalescing.
C-32	Coalescing	Same rule/generation and normal conditions pass	Coalescing remains available.
C-33	Update	No-rule hostname becomes protected	Stale cache/session cannot bypass new rule.
C-34	Update	Old CA removed after overlap	Old sessions/tickets invalidated; new CA works.

ID	Area	Scenario	Expected result
C-35	Update	Emergency SPKI revocation	Affected sessions close immediately.
C-36	Update	Malformed new bundle	Last-known-good remains active.
C-37	Update	Replay older signed generation	Rejected as rollback.
C-38	Expiry	Bundle expired; protected host	Fail closed.
C-39	Expiry	Bundle expired; unrelated host	Baseline browsing remains available.
C-40	Surface	WebSocket reconnect after tightening	New handshake enforces current rule.
C-41	Surface	Service-worker background fetch after tightening	Current rule enforced.
C-42	Resumption	0-RTT/ticket from old generation for protected host	Early data blocked and stale resumption rejected.
C-43	Platform	macOS Keychain private root, unmatched host	TCP and QUIC success.
C-44	Platform	macOS Keychain interception root, protected host	Reject.
C-45	Platform	Linux local/enterprise root, unmatched host	TCP and QUIC success.
C-46	Platform	Linux local/enterprise root, protected host	Reject unless approved.
C-47	Delegation	dev delegate signs app.dev.scone.cloud	Accepted within delegation.
C-48	Delegation	dev delegate signs prod.scone.cloud	Rejected.
C-49	Rollover	Old+new CA overlap	Both accepted; unrelated CA rejected.
C-50	Recovery	Offline root signs emergency replacement	Accepted at higher generation; old revoked.

D.1 Release gate

Every stable browser build must pass C-01 through C-50 on Linux and all platform-independent tests on macOS; C-43 and C-44 are mandatory on macOS, while C-45 and C-46 are mandatory on Linux. Any expected-result change requires a whitepaper/security-design revision, not merely a test update.

Appendix E. Command and release runbook

E.1 Calculate CA identifiers

```
# Certificate SHA-256 fingerprint
openssl x509 -in ca.pem -noout -fingerprint -sha256

# Base64 SHA-256 SPKI identifier used by DomainTrustPolicy
SPKI=$(openssl x509 -in ca.pem -pubkey -noout |
 openssl pkey -pubin -outform DER |
 openssl dgst -sha256 -binary |
 openssl base64 -A)
printf 'sha256/%s\n' "$SPKI"

# Base64 DER when the signed bundle supplies a private anchor
openssl x509 -in ca.pem -outform DER | base64 | tr -d '\n'
printf '\n'
```

E.2 Verify CA constraints before onboarding

```
openssl x509 -in ca.pem -noout -text
# Review at minimum:
#   Basic Constraints: critical, CA:TRUE
#   Key Usage: critical, Certificate Sign, CRL Sign as intended
#   Name Constraints: critical, permitted DNS subtree scone.cloud
#   Signature/public-key algorithms and validity

# Verify a representative chain with an explicit hostname:
openssl verify -show_chain -verify_hostname api.scone.cloud \
 -CAfile ca.pem -untrusted intermediate.pem leaf.pem
```

E.3 Build and focused-test loop

```
# Discover current targets rather than assuming labels:
gn refs out/DomainTrust net/cert/<changed-file>
gn desc out/DomainTrust //net:net_unittests deps --tree

# Representative focused build:
autoninja -C out/DomainTrust \
 chrome net_unittests unit_tests browser_tests

# Run selected tests; exact names are implementation-defined:
out/DomainTrust/net_unittests \
 --gtest_filter='*DomainTrust*:*ProofVerifierChromium*'
out/DomainTrust/browser_tests \
 --gtest_filter='*DomainTrustPolicy*'
```

E.4 Linux packaging release checklist

- Checkout the signed release branch recursively and verify submodule commits.
- Build with the pinned container image using scripts/docker-build.sh -c.
- Run complete conformance lab against the exact packaged binary.
- Run scripts/package.sh or the pinned containerized packaging command.
- Generate SBOM, licenses, symbols, package hashes and signed provenance.
- Sign update metadata and publish to dev; promote to pilot only after clean installation/update tests.
- Record policy-schema compatibility and minimum supported bundle version.

E.5 macOS signing/notarization checklist

- Build the selected architecture from a clean pinned worker.
- Verify every nested helper/framework/library is signed with the intended identity and entitlements.
- Run `codesign --verify --deep --strict --verbose=4 <App>` and product-specific tests.
- Create notarization archive, submit with `xcrun notarytool`, wait for success and staple.
- Run `spctl --assess --type execute --verbose=4 <App>` on a clean machine.
- Package the notarized app, sign update metadata and perform install/update/rollback-protection tests.
- Archive notary submission ID, signing certificate chain, hashes, SBOM and provenance.

E.6 CA rotation operator checklist

- Validate current-authority rollover proof and next-key possession.

- Validate next CA DER/SPKI, constraints and scope.
- Publish audit/pilot bundle containing both old and next CA.
- Run C-13 through C-35 and C-49 on pilot clients, including HTTP/3.
- Confirm fleet adoption threshold and service readiness.
- Switch server issuance/chains to next CA; monitor failures.
- Publish tightening bundle removing old CA and verify session invalidation.
- Revoke/offline old CA and close change record.

E.7 Incident triage commands

```
# Capture server chain and negotiated protocol
openssl s_client -connect api.scone.cloud:443 \
  -servername api.scone.cloud -showcerts -verify_return_error </dev/null

# In the browser:
# 1. Open chrome://policy and confirm bootstrap source/status.
# 2. Open the enterprise domain-trust diagnostics page.
# 3. Capture chrome://net-export with sensitive-data policy observed.
# 4. Record browser version, bundle generation, rule ID and anchor SPKI.
# 5. Do not bypass the certificate interstitial.
```

Appendix F. References

[1] ungoogled-chromium, "Design": configuration, source processors, GNU Quilt patches and packaging architecture.
https://github.com/ungoogled-software/ungoogled-chromium/blob/master/docs/design.md

[2] ungoogled-chromium, "Building ungoogled-chromium": integration and platform-repository guidance.
https://github.com/ungoogled-software/ungoogled-chromium/blob/master/docs/building.md

[3] Chromium, "Checking out and building Chromium on Linux."
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/docs/linux/build_instructions.md

[4] Chromium, "Checking out and building Chromium for Mac."
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/docs/mac_build_instructions.md

[5] ungoogled-chromium-portablelinux repository and build/package scripts. https://github.com/ungoogled-software/ungoogled-chromium-portablelinux

[6] ungoogled-chromium-macos repository, build, signing/notarization and patch-refresh workflow. https://github.com/ungoogled-software/ungoogled-chromium-macos

[7] Chromium source, ProofVerifierChromium and QUIC certificate post-verification checks.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/net/quic/crypto/proof_verifier_chromium.cc

[8] Chromium source, ProfileNetworkContextService enterprise certificate-policy processing.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/chrome/browser/net/profile_network_context_service.cc

[9] Chromium source, SSLClientSocketImpl TCP TLS certificate verification.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/net/socket/ssl_client_socket_impl.cc

[10] Chromium source, CertVerifyResult.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/net/cert/cert_verify_result.h

[11] Chromium source, SystemTrustStore abstraction.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/net/cert/system_trust_store.h

[12] BoringSSL PKI source, TrustStore interface. https://boringssl.googlesource.com/boringssl/+/refs/heads/master/pki/trust_store.h

[13] Chromium source, certificate verifier service.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/services/cert_verifier/

[14] Chromium component updater source and documentation.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/components/component_updater/

[15] Chromium policy template for CACertificatesWithConstraints.
https://chromium.googlesource.com/chromium/src/+/refs/heads/main/components/policy/resources/templates/policy_definitions/CertificateManagement/CACertificatesWithConstraints.yaml

[16] Brave browser source repository. https://github.com/brave/brave-browser

[17] Cromite source repository. https://github.com/uazo/cromite

[18] Thorium browser source repository. https://github.com/Alex313031/thorium

[19] Helium browser source repository. https://github.com/imputnet/helium

[20] GrapheneOS Vanadium source repository. https://github.com/GrapheneOS/Vanadium

[21] IETF RFC 5280, Internet X.509 Public Key Infrastructure Certificate and CRL Profile. https://www.rfc-editor.org/rfc/rfc5280

[22] IETF RFC 6125, Representation and Verification of Service Identity. https://www.rfc-editor.org/rfc/rfc6125

[23] IETF RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3. https://www.rfc-editor.org/rfc/rfc8446

[24] IETF RFC 9000, QUIC: A UDP-Based Multiplexed and Secure Transport. https://www.rfc-editor.org/rfc/rfc9000

[25] IETF RFC 9114, HTTP/3. https://www.rfc-editor.org/rfc/rfc9114

[26] IETF RFC 9052, CBOR Object Signing and Encryption (COSE): Structures and Process. https://www.rfc-editor.org/rfc/rfc9052

[27] IETF RFC 7515, JSON Web Signature (JWS). https://www.rfc-editor.org/rfc/rfc7515

[28] Public Suffix List. https://publicsuffix.org/

Document conclusion

Implementation result The proposed fork can enforce an explicit CA for selected DNS subtrees while retaining normal Chromium public and locally added private trust for all unmatched domains. QUIC remains enabled: its extra unknown-root decision consumes explicit verifier trust-source data, while the same pre-path domain filter protects constrained domains over TCP TLS and HTTP/3.